Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Reimplementácia prieskumníka štruktúr pre logiku prvého rádu

Bakalárska práca

2025 Jozef Filip

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Reimplementácia prieskumníka štruktúr pre logiku prvého rádu

Bakalárska práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Katedra aplikovanej informatiky
Školiteľ:	Mgr. Ján Kľuka, PhD

Bratislava, 2025 Jozef Filip

83077893

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Jozef Filip
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Reimplementácia prieskumníka štruktúr pre logiku prvého rádu
Reimplementation of the explorer of first-order logic structures

Anotácia: V rámci série predchádzajúcich bakalárskych prác (Cifra, 2018; Baluch, 2020; Tóth, 2021) vznikla interaktívna webová aplikácia, ktorá umožňuje skúmať pravdivosť formúl a hodnoty termov v konkrétnych konečných štruktúrach pre jazyky logiky prvého rádu. Vnútorný návrh aplikácie však vznikol v čase, keď použité frameworky neposkytovali súčasné prostriedky, je neprehľadný a ďalšie úpravy sú prácne a prinášajú riziko vzniku ťažko odhaliteľných chýb. Zároveň sme zistili, že okamžitá spätná väzba, ktorú prieskumník poskytuje, nie je z pedagogického hľadiska vždy vhodná. Potrebovali by sme preto posilniť úlohu Henkinovej-Hintikkovej hry, ktorá podporuje hlbšie pochopenie (ne)pravdivosti formúl.

Cieľ: Reimplementovať jadro prieskumníka štruktúr: zadanie jazyka, tvorbu štruktúry a uzavretých formúl.
Reimplementovať Henkinovu-Hintikkovu hru ako certifikáciu pravdivosti uzavretých formúl.

Literatúra:
Cifra, Milan. Prieskumník sémantiky logiky prvého rádu. Bakalárska práca. Bratislava : Univerzita Komenského, 2018.
Baluch, Miroslav. Prieskumník grafových štruktúr pre logiku prvého rádu. Bakalárska práca. Bratislava : Univerzita Komenského, 2020.
Tóth, Richard. Henkinova-Hintikkova hra v prieskumníku štruktúr. Bakalárska práca. Bratislava : Univerzita Komenského, 2021.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 28.10.2024

Dátum schválenia: 30.05.2025

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Poďakovanie: Týmto chcem poďakovať školiteľovi práce, Mgr. Jánovi Kľukovi, PhD., za cenné rady a pravidelné konzultácie, ktoré ma motivovali, aby som napredoval v našej práci. Ďalej sa chcem poďakovať rodine, priateľke a kamarátom za podporu.

Abstrakt

Prieskumník štruktúr je webová aplikácia, ktorá pomáha študentom lepšie pochopiť pravdivosť formúl a hodnoty termov v konkrétnej štruktúre pre jazyk logiky prvého rádu. V rámci našej bakalárskej práce sme vytvorili novú verziu aplikácie, ktorá obsahuje najpoužívanejšiu funkcionalitu predchádzajúcej verzie vrátane posilnenia úlohy Henkinovej-Hintikkovej hry. Spravili sme analýzu predchádzajúcej verzie aplikácie, na základe ktorej sme našli problémy. Vytvorili sme návrh, v ktorom sme dané problémy odstránili. Podľa návrhu sme implementovali novú verziu aplikácie, kde sme využili novšie verzie frameworkov a zjednodušili a sprehľadnili sme štruktúru zdrojového kódu oproti predchádzajúcej verzii. Implementovali sme našu aplikáciu tak, aby poskytovala možnosť ju integrovať do Logického hárku. Nakoniec sme na novej verzii aplikácie vykonali menšie testovanie použiteľnosti, ktorého výsledky sme následne analyzovali.

Kľúčové slová: logika prvého rádu, webová aplikácia, Henkinova-Hintikkova hra, vyhodnotenie formúl

Abstract

Structure Explorer is a web application that helps students better understand the truth of formulas and the values of terms within a specific structure for the language of first-order logic. As part of our bachelor's thesis, we developed a new version of the application, which includes the most frequently used functionality from the previous version while enforcing the Henkin-Hintikka game. We analyzed the previous version of the application and identified several issues. Based on this analysis, we designed a new solution that addressed these problems. According to this design, we implemented a new version of the application using the latest features that are present in frameworks and improved and simplified the source code structure compared to the previous version. We also implemented the application in such a way that allows integration with the Logic workbook. Finally, we performed a small usability test on the new version of the application and analyzed the results.

Keywords: first-order logic, web application, Henkin-Hintikka game, formula evaluation

Obsah

Úvod **1**

1 Východiská práce **3**

1.1 Logika prvého rádu . . . 3

1.1.1 Syntax . . . 3

1.1.2 Sémantika . . . 5

1.2 Henkinova-Hintikkova hra . . . 6

1.2.1 Pravidlá hry . . . 6

1.2.2 Ukážka hry . . . 7

1.3 Použité technológie . . . 9

1.3.1 HTML, CSS a Bootstrap . . . 9

1.3.2 TypeScript . . . 9

1.3.3 React . . . 9

1.3.4 Redux a Redux Toolkit . . . 10

1.3.5 Vite . . . 10

1.4 Súvisiace aplikácie . . . 11

1.4.1 Logický hárok . . . 11

1.4.2 Prieskumník štruktúr . . . 11

2 Analýza existujúceho riešenia **13**

2.1 Architektúra pôvodnej aplikácie . . . 13

2.1.1 Funkcionalita . . . 13

2.1.2 Štruktúra modulov zdrojového kódu . . . 15

2.1.3 Dátový model na vyhodnocovanie výrazov . . . 17

2.1.4 Štruktúra stavu . . . 17

2.2 Nedostatky a problémy pôvodnej aplikácie . . . 18

2.2.1 Miešanie JavaScriptu a TypeScriptu . . . 18

2.2.2 Kombinácia funkčných komponentov a triednych komponentov . 19

2.2.3 Stará verzia Reduxu a absencia Redux Toolkitu . . . 19

2.2.4 Okamžitá odozva splniteľnosti formuly . . . 19

2.2.5 Zriedkavé využívanie Henkinovej-Hintikkovej hry . . . 19

3 Návrh novej aplikácie **21**
3.1 Požiadavky 21
3.1.1 Funkcionalita 21
3.1.2 Požívateľské rozhranie 22
3.2 Interpretácia Henkinovej-Hintikkovej hry 22
3.2.1 Označené formuly 22
3.2.2 Typy označených formúl a ich podformuly 23
3.2.3 Nové pravidlá Henkinovej-Hintikkovej hry 24
3.3 Návrh stavu aplikácie 26
3.4 Dátový model na vyhodnocovanie výrazov 26

4 Implementácia novej aplikácie **29**
4.1 Inicializácia a súborová štruktúra 29
4.2 Dátový model na vyhodnocovanie výrazov 30
4.3 Stav aplikácie 30
4.3.1 Stav formulas 30
4.3.2 Stav language 31
4.3.3 Stav structure 31
4.3.4 Stav variables 31
4.3.5 Reaktivita a práca so stavom 31
4.4 Implementácia Henkinovej-Hintikkovej hry 31
4.4.1 Selektory pre Henkinovu-Hintikkovu hru 32
4.4.2 Komponenty Henkinovej-Hintikkovej hry 33
4.4.3 Reducery pre Henkinovu-Hintikkovu hru 34
4.5 Implementácia knižnice pre Logický hárok 36

5 Testovanie **39**

Záver **41**

Zoznam obrázkov

2.1 Množinové zobrazenie pôvodného Prieskumníka štruktúr 14
2.2 Grafové zobrazenie pôvodného Prieskumníka štruktúr 14
2.3 Spätná väzba pri zápornej arite predikátu 14
2.4 Tabuľkový pohľad interpretácie predikátu 15
2.5 Databázový pohľad interpretácie predikátu 15
2.6 Prehra Henkinovej-Hintikkovej hry pri správnom odhade 16

3.1 Triedny diagram dátového modelu 25

4.1 Voľby v Henkinovej-Hintikkovej hre 35
4.2 Naša aplikácia v lokálnom Logickom hárku 37

5.1 Prieskumník štruktúr s predvyplnenými formulami zo zadania 39
5.2 Dotazník na vyhodnotenie použiteľnosti 40

Zoznam tabuliek

3.1 Označené formuly typu α 23
3.2 Označené formuly typu β 23
3.3 Označené formuly typu γ 24
3.4 Označené formuly typu δ 24

Úvod

Prieskumník štruktúr je webová aplikácia, ktorá sa využíva na predmete Logika pre informatikov. Aplikácia pomáha študentom lepšie pochopiť pravdivosť formúl a hodnoty termov v konkrétnej štruktúre pre jazyk logiky prvého rádu. Návrh aplikácie ale vznikol v čase, keď ešte využívané technológie neposkytovali takú funkcionalitu, akú poskytujú dnes. Keďže aplikácia postupne vznikala vo viacerých bakalárskych prácach [1, 2, 3], tak sa jej priečinková štruktúra aj zdrojový kód stali neprehľadnými. To komplikuje opravu chýb a budúce rozširovanie aplikácie. Zároveň vyučujúci dlhodobým sledovaním zistili, že spôsob, akým je implementovaná Henkinova-Hintikkova hra – poskytuje okamžitú spätnú väzbu – nie je ideálny z pedagogického hľadiska, keďže pri okamžitom vyhodnotení často niektorí študenti sledujú len výsledok a nezamyslia sa, prečo je formula splnená. Prieskumník štruktúr síce obsahuje Henkinovu-Hintikkovu hru [3, 4], ktorá overuje hlbšie pochopenie splnenia formúl, ale Prieskumník štruktúr nemotivuje k jej využívaniu.

Cieľom našej práce je vytvoriť novú verziu aplikácie, ktorá tieto nedostatky odstráni. Spravili sme analýzu predchádzajúcej verzie aplikácie – analyzovali sme jej funkcionalitu, súborovú štruktúru, štruktúru jej centralizovaného stavu a jej nedostatky. Následne sme z tejto analýzy spravili návrh novej verzie aplikácie – teda požiadavky, ktoré chceme, aby spĺňala. Navrhli sme novú štruktúru stavu aplikácie a mierne sme upravili dátový model. Taktiež sme zadefinovali vlastnú interpretáciu Henkinovej-Hintikkovej hry pomocou označených formúl a aj nové pravidlá, ktoré z nej vyplývajú. Následne sme samotnú aplikáciu implementovali podľa vytvoreného návrhu, vrátane vytvorenia verzie, ktorú je možné vložiť do aplikácie Logicky hárok [5], ktorá integruje viacero aplikácií na podporu výučby logiky do ucelenej aplikácie, ktorá pripomína pracovné hárky. Na záver sme podnikli menšie testovanie použiteľnosti, po ktorom študenti vyplnili krátky dotazník na vyhodnotenie použiteľnosti aplikácie.

V prvej kapitole si popíšeme východiská práce – logiku prvého rádu, Henkinovu-Hintikkovu hru aj s ukážkou jej priebehu, použité technológie v našej práci a nakoniec aplikácie, ktoré priamo súvisia s našou aplikáciou. V druhej kapitole budeme analyzovať existujúcu aplikáciu – konkrétne jej funkcionalitu, stav, dátový model, ako sa členia jej súbory a na záver jej nedostatky, ktoré chceme odstrániť. V tretej kapitole sa pozrieme na návrh novej aplikácie a zadefinujeme novú interpretáciu Henkinovej-Hintikkovej hry

pomocou označených formúl spolu s novými odvodenými pravidlami pre ňu. V štvrtej kapitole sa zameriame na samotnú implementáciu podľa návrhu z tretej kapitoly a implementáciu knižničnej verzie našej aplikácie. Piatu kapitolu venujeme testovaniu použiteľnosti našej novej aplikácie a výsledkom testovania.

Kapitola 1

Východiská práce

V tejto kapitole si popíšeme všetky potrebné pojmy z logiky prvého rádu, Henkinovu-Hintikkovu hru, použité technológie a nakoniec si opíšeme súvisiace aplikácie.

1.1 Logika prvého rádu

V tejto podkapitole si popíšeme základné pojmy z logiky prvého rádu potrebné na využívanie našej webovej aplikácie. Použité definície sú z poznámok k prednáškam k predmetu Logika pre informatikov [6], ktoré vychádzajú z knihy Logika: neúplnost, složitost a nutnost [7].

Logika prvého rádu predstavuje formálny systém, ktorý umožňuje vyjadrovať vlastnosti objektov a vzťahy medzi nimi pomocou kvantifikátorov a predikátov.

1.1.1 Syntax

Symboly logiky prvého rádu, nazývané aj *jazyk* $\mathcal{L}$ sú:

- symboly indivíduových premenných $\mathcal{V}_{\mathcal{L}}$ označujú objekty alebo hodnoty, ktoré nie sú dopredu známe
- mimologické symboly:
  - spočítateľná množina symbolov konštánt $\mathcal{C}_{\mathcal{L}}$ – konštanty predstavujú konkrétne objekty alebo hodnoty
  - spočítateľná množina predikátových symbolov $\mathcal{P}_{\mathcal{L}}$ – predikáty predstavujú vlastnosti a vzťahy
  - spočítateľná množina funkčných symbolov $\mathcal{F}_{\mathcal{L}}$ – funkčné symboly predstavujú jednoznačne určené symboly a vzťahy
- logické symboly:

  - logické spojky: $\neg$, $\wedge$, $\vee$, $\rightarrow$
  - symbol rovnosti: $\doteq$,
  - existenčný kvantifikátor $\exists$ a všeobecný kvantifikátor $\forall$;

- pomocné symboly (,) a , (zátvorky a čiarka)

Všetky tieto množiny sú navzájom disjunktné. Každému symbolu $S \in \mathcal{P}_{\mathcal{L}} \cup \mathcal{F}_{\mathcal{L}}$ je priradená arita $ar(S) \in \mathbb{N}^+$, ktorá sa bude písať ako horný index, napríklad: p^2

Množina $\mathcal{T}_{\mathcal{L}}$ *termov* jazyka logiky prvého rádu $\mathcal{L}$ je najmenšia množina postupností symbolov jazyka $\mathcal{L}$, pre ktorú platí:

- každý symbol premennej $x \in \mathcal{V}_{\mathcal{L}}$ je termom;
- každý symbol konštanty $c \in \mathcal{C}_{\mathcal{L}}$ je termom;
- ak f je funkčný symbol s aritou n a $t_1, \ldots, t_n$ sú termy, potom aj $f(t_1, \ldots, t_n)$ je termom.

Atomickými formulami nazývame všetky rovnostné a predikátové atómy jazyka $\mathcal{L}$:

- Každú postupnosť symbolov $t_1 \doteq t_2$ nazývame **rovnostný atóm**.
- Ak P je predikátový symbol s aritou n a $t_1, \ldots, t_n$ sú termy, potom postupnosť symbolov $P(t_1, \ldots, t_n)$ nazývame **predikátový atóm** jazyka $\mathcal{L}$.

Množinu všetkých atómov jazyka $\mathcal{L}$ označujeme ako $\mathcal{A}_{\mathcal{L}}$.

Množina $\mathcal{E}_{\mathcal{L}}$ *formúl* jazyka logiky prvého rádu $\mathcal{L}$ je najmenšia množina postupností symbolov jazyka $\mathcal{L}$, pre ktorú platia nasledovné pravidlá:

- Všetky atomické formuly z $\mathcal{A}_{\mathcal{L}}$ sú formulami.
- Ak A je formula, tak aj $\neg A$ je formula (negácia A).
- Ak A a B sú formuly, tak aj $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, $(A \leftrightarrow B)$ sú formuly.
- Ak x je indivíduová premenná a A je formula, tak aj $\exists x\, A$ a $\forall x\, A$ sú formuly.

Označenie ekvivalencie $(A \leftrightarrow B)$ považujeme ako skratku za formulu $((A \rightarrow B) \wedge (B \rightarrow A))$.

1.1.2 Sémantika

Štruktúrou pre jazyk $\mathcal{L}$ nazývame dvojicu $\mathcal{M} = (D, i)$, kde:

- D je neprázdna množina – **doména** štruktúry $\mathcal{M}$;
- i je zobrazenie – **interpretačná funkcia** štruktúry $\mathcal{M}$, ktoré:
  - každému symbolu konštanty c jazyka $\mathcal{L}$ priraďuje prvok $i(c) \in D$;
  - každému funkčnému symbolu f jazyka $\mathcal{L}$ s aritou n priraďuje funkciu $i(f) : D^n \to D$;
  - každému predikátovému symbolu P jazyka $\mathcal{L}$ s aritou n priraďuje množinu $i(P) \subseteq D^n$.

Nech $\mathcal{M} = (D, i)$ je štruktúra a e je ohodnotenie premenných.

Ohodnotenie indivíduových premenných v štruktúre $\mathcal{M}$ je funkcia $e : \mathcal{V}_{\mathcal{L}} \to D$, ktorá priraďuje každej premennej prvok z domény. V štruktúre $\mathcal{M}$ s ohodnotením e sa vieme pýtať na hodnotu termu v danej štruktúre pri danom ohodnotení alebo na pravdivosť formuly X v danej štruktúre pri danom ohodnotení. Nech x je individuová premenná z $\mathcal{L}$ a m je prvok z D. Označenie $e(x/m)$ označuje ohodnotenie e', pre ktoré platí:

- $e'(x) = m$,
- $e'(y) = e(y)$, ak y je iná premenná ako x.

Hodnotou termu t v štruktúre $\mathcal{M}$ pri ohodnotení e je prvok $t^{\mathcal{M}}[e] \in D$ určený nasledovne:

- $x^{\mathcal{M}}[e] = e(x)$, ak x je premenná;
- $a^{\mathcal{M}}[e] = i(a)$, ak a je konštanta;
- $f(t_1, \ldots, t_n)^{\mathcal{M}}[e] = i(f)(t_1^{\mathcal{M}}[e], \ldots, t_n^{\mathcal{M}}[e])$, ak $t_1, \ldots, t_n$ sú termy.

Nech $\mathcal{M} = (D, i)$ je štruktúra a e je ohodnotenie premenných.
Relácia „*štruktúra $\mathcal{M}$ spĺňa formulu A pri ohodnotení e*“ (značíme $\mathcal{M} \models A[e]$) je definovaná rekurzívne nasledovne:

- $\mathcal{M} \models t_1 = t_2[e]$ práve vtedy, keď $t_1^{\mathcal{M}}[e] = t_2^{\mathcal{M}}[e]$;
- $\mathcal{M} \models P(t_1, \ldots, t_n)[e]$ práve vtedy, keď $(t_1^{\mathcal{M}}[e], \ldots, t_n^{\mathcal{M}}[e]) \in i(P)$;
- $\mathcal{M} \models \neg A[e]$ práve vtedy, keď $\mathcal{M} \not\models A[e]$;
- $\mathcal{M} \models (A \wedge B)[e]$ práve vtedy, keď $\mathcal{M} \models A[e]$ a zároveň $\mathcal{M} \models B[e]$;

- $\mathcal{M} \models (A \vee B)[e]$ práve vtedy, keď $\mathcal{M} \models A[e]$ alebo $\mathcal{M} \models B[e]$;
- $\mathcal{M} \models (A \rightarrow B)[e]$ práve vtedy, keď $\mathcal{M} \not\models A[e]$ alebo $\mathcal{M} \models B[e]$;
- $\mathcal{M} \models \exists x\, A[e]$ práve vtedy, keď existuje $m \in D$ také, že $\mathcal{M} \models A[e(x/m)]$;
- $\mathcal{M} \models \forall x\, A[e]$ práve vtedy, keď pre každé $m \in D$ platí $\mathcal{M} \models A[e(x/m)]$.

Pre všetky arity $n > 0$, všetky predikátové symboly P s aritou n, všetky termy $t_1, \ldots, t_n$, všetky premenné x a všetky formuly A, B.

Formula A je *uzavretá* práve vtedy, keď žiadna premenná nie je voľná v A. Teda všetky výskyty premenných sú v oblasti platnosti nejakého kvantifikátora.

Nech A je uzavretá formula jazyka $\mathcal{L}$ a nech $\mathcal{M}$ je štruktúra pre $\mathcal{L}$. Formula A je pravdivá v štruktúre $\mathcal{M}$ práve vtedy, keď M spĺňa formulu X v každom ohodnotení e.

1.2 Henkinova-Hintikkova hra

V tejto podkapitole si vysvetlíme pravidlá hry a znázorníme si jej priebeh na ukážke.

1.2.1 Pravidlá hry

Henkinova-Hintikkova hra [4] je spôsob, ako určiť, či je formula splnená v danej štruktúre, alebo nie. Je to alternatíva k Tarského definícii splnenia. Odohráva sa medzi dvoma hráčmi – Študent (v knihe Ja) a Svet (v knihe Príroda), kde sa Študent snaží formulu dokázať a Svet sa ju snaží vyvrátiť. V hre predpokladáme pevne daný jazyk $\mathcal{L}$ so štruktúrou $\mathcal{M} = (D, i)$, kde D je doména a i je interpretačná funkcia. Nech $G(F)$ je krok hry G pri formule F. Hra má nasledujúce pravidlá:

a. Ak F je pravdivá atomická formula, tak vyhral Študent a Svet prehral. Ak F je nepravdivá atomická formula, tak vyhral Svet a Študent prehral.

b. Ak F je konjunkcia $(F_1 \wedge F_2)$, tak hra začína tým, že si vyberá Svet jednú z možností a hra pokračuje s danou možnosťou – $G(F_1)$ alebo $G(F_2)$.

c. Ak F je disjunkcia $(F_1 \vee F_2)$, tak hra začína tým, že si vyberá Študent jednú z možností a hra pokračuje s danou možnosťou – $G(F_1)$ alebo $G(F_2)$.

d. Ak F je všeobecná kvantifikácia $(\forall x F[x])$, tak hra začína tým, že si vyberá Svet jeden prvok d z domény $d \in D$, ktorý pomenuje napríklad b. Hra následne pokračuje s danou formulou, v ktorej premennú x nahradíme novým menom b – $G(F[b])$.

e. Ak F je existenčná kvantifikácia ($\exists x F[x]$), tak hra začína tým, že si vyberá Študent jeden prvok d z domény $d \in D$, ktorý pomenuje napríklad b. Hra následne pokračuje s danou formulou, v ktorej premennú x nahradíme novým menom b – $G(F[b])$.

f. Ak F je negácia ($\neg F$), tak sa roly hráčov vymenia – ak by vyberal Svet, tak vyberá Študent a naopak.

Vyššie uvedené pravidlá sa rekurzívne opakujú, kým nenastane pravidlo a., kde hra končí.

Pravdivosť formuly F Hintikka definuje aj ako existenciu výhernej stratégie pre Študenta, t.j. existuje také poradie krokov (aplikovanie pravidiel), kde nech hrá Svet hocijak, tak vždy prehrá. Naopak, nepravdivosť formuly F vieme definovať ako existenciu výhernej stratégie pre Svet. Hintikka dokázal, že takáto definícia pravdivosti je ekvivalentná s Tarského definíciou.

Ak je Študentov počiatočný predpoklad, že formula F je nepravdivá, tak majú Študent a Svet od začiatku vymenené roly.

Môžeme vidieť, že nie sú definované pravidlá pre implikáciu ani ekvivalenciu. Tieto pravidlá si vieme odvodiť, keďže implikácia ($F_1 \rightarrow F_2$) je ekvivalentná s disjunkciou ($\neg F_1 \vee F_2$) a vieme použiť pravidlo c.) a ekvivalencia je v časti 1.1.1 definovaná ako skratka konjunkcie dvoch implikácií, takže vieme použiť pravidlo b.). Na rozdiel od Tarského definície, kde sa pri vyhodnocovaní kvantifikátorov používa ohodnotenie, v Henkinovej-Hintikkovej hre sa vyberajú nové mená, ktoré sa substituujú do formuly.

1.2.2 Ukážka hry

Nech $\mathcal{L}$ je jazyk logiky prvého rádu, ktorý je zložený z konštánt $\mathcal{C}_{\mathcal{L}} = \{\}$, predikátových symbolov $\mathcal{P}_{\mathcal{L}} = \{dobry^1, chlapec^1, auticko^1, motorka^1, dostane^2\}$ a funkčných symbolov $\mathcal{F}_{\mathcal{L}} = \{\}$. Nech štruktúra $\mathcal{M}$ jazyka $\mathcal{L}$ má definovanú doménu $D = \{1, 2, 3, 4\}$ a interpretačnú funkciu i nasledovne:

- $i(dobry) = \{1\}$,
- $i(chlapec) = \{1\}$,
- $i(auticko) = \{2\}$,
- $i(motorka) = \{3\}$,
- $i(dostane) = \{(1, 2), (1, 3)\}$

Hru si ukážeme na formule:

$$F = \forall x((dobry(x) \wedge chlapec(x)) \rightarrow \exists y((auticko(y) \vee motorka(y)) \wedge \neg dostane(x, y)))$$

Hra začína výberom Študenta a ten si myslí, že formula je pravdivá a to ide dokázať. Keďže formula je všeobecne kvantifikovaná, tak sa uplatní pravidlo d. a Svet si vyberá prvok z domény. Nech si Svet vyberie prvok 1 a pomenuje ho x_1. Po substitúcii $F\{x \mapsto x_1\}$ dostaneme novú formulu:

$$F_1 = (dobry(x_1) \wedge chlapec(x_1)) \rightarrow \exists y((auticko(y) \vee motorka(y)) \wedge \neg dostane(x_1, y))$$

Táto formula je implikácia, ktorá je ekvivalentná s disjunkciou a po prepísaní vyzerá nasledovne:

$$F_1' = \neg(dobry(x_1) \wedge chlapec(x_1)) \vee \exists y((auticko(y) \vee motorka(y)) \wedge \neg dostane(x_1, y))$$

Pretože táto formula je disjunkcia, tak podľa pravidla c. si vyberá Študent, ktorá priama podformula je podľa neho pravdivá. Nech si Študent vyberie podformulu:

$$F_2 = \exists y((auticko(y) \vee motorka(y)) \wedge \neg dostane(x_1, y)),$$

s ktorou hra pokračuje. Táto formula je existenčne kvantifikovaná, takže podľa pravidla e. si vyberá Študent, aký prvok z domény dosadí namiesto y, aby formula platila. Nech si Študent vyberie prvok 1 a pomenuje ho y_1. Po substitúcii $F_2\{y \mapsto y_1\}$ dostaneme novú formulu:

$$F_3 = (auticko(y_1) \vee motorka(y_1)) \wedge \neg dostane(x_1, y_1)$$

ktorá je konjunkcia a podľa pravidla b. si vyberá jednu z priamych podformúl Svet. V tomto kroku si môže Svet vybrať len podformulu $(auticko(y_1) \vee motorka(y_1))$, pretože ak by si vybral $\neg dostane(x_1, y_1)$, tak by automaticky prehral, pretože $(1, 1) \notin i(dostane)$. Tu môžeme vidieť, že na výberoch pre Svet záleží – nemôže spraviť nesprávny výber, pretože by inak vedel Študent dokázať vyhrať, aj keď je formula nepravdivá.

Hra ďalej pokračuje formulou $F_4 = (auticko(y_1) \vee motorka(y_1))$, čo je disjunkcia, takže sa znova uplatňuje pravidlo c., kde si vyberá Študent priamu podformulu, o ktorej si myslí, že je pravdivá. Nech si Študent vyberie $auticko(y_1)$, čo je atomická formula a podľa pravidla a. prehráva. V tomto prípade nezáleží, čo si Študent vyberie, v oboch prípadoch prehrá, pretože $1 \notin i(auticko)$ a ani $1 \notin i(motorka)$. Dokonca ak Svet hrá bezchybne, tak ak Študentov počiatočný odhad bol nesprávny, tak nikdy nemôže vyhrať.

Na záver si ukážeme na formule

$$G = \exists x \neg(dobry(x) \wedge chlapec(x))$$

prípad, keď Študent má výhernú stratégiu. Jazyk aj štruktúra ostávajú rovnaké ako pri formule F. Nech študentov počiatočný odhad je, že formula je pravdivá. Keďže vo formule máme existenčný kvantifikátor, tak použijeme pravidlo e., podľa ktorého vyberá Študent prvok z domény, pre ktorý táto formula platí. Nech si študent vyberie prvok 2 a pomenuje ho x_1. Po substitúcii dostaneme formulu

$$G_1 = \neg(dobry(x_1) \wedge chlapec(x_1))$$

keďže formula je negovaná, podľa pravidla f. sa v ďalšom kroku roly menia. Ďalej je formula konjunkciou a podľa pravidla b. by mal vyberať priamu podformulu Svet, ale kvôli negácii ju vyberá Študent. Nech si študent vyberie formulu $dobry(x_1)$. Týmto Študent vyhráva, keďže $2 \notin i(dobry)$. Ak by si ale v prvom výbere, teda pri formule G vybral ako prvok domény 1, tak by prehral. Tu je vidno, že Študent musí pozorne vyberať správne možnosti, pretože ak bude nepozorný môže ľahko prehrať.

1.3 Použité technológie

V tejto podkapitole si popíšeme použité technológie v našej práci.

1.3.1 HTML, CSS a Bootstrap

Technológie ako HTML [8] a CSS [9] sú základné technológie na tvorbu webových aplikácií. HTML slúži na štruktúrovanie obsahu, zatiaľ čo CSS slúži na úpravu jeho vzhľadu. Na štýlovanie vzhľadu a responzívny dizajn aplikácie využívame framework Bootstrap, ktorý má preddefinované triedy pre jednoduchú tvorbu moderného dizajnu bez potreby písania vlastného CSS kódu.

1.3.2 TypeScript

TypeScript [10] predstavuje rozšírenie jazyka JavaScript, ktoré prináša viacero výhod. Najväčšou výhodou je „silné“ typovanie, ktoré často odhalí chyby v kóde pred spustením samotného programu. Napriek tomu však typový systém nie je striktne obmedzujúci – TypeScript dokáže celkom presne inferovať typy na základe deklarácií, priradení alebo použitia. Keďže sa TypeScript pred spustením aplikácie prekladá do JavaScriptu, je možné ho použiť všade, kde by sa dal použiť JavaScript.

1.3.3 React

React [11] je JavaScriptová knižnica, ktorá sa používa na tvorbu používateľského rozhrania pre webové aplikácie. Základom Reactu sú komponenty – každý komponent

popisuje konkrétnu časť rozhrania. Každý komponent je definovaný ako JavaScriptová funkcia, prípadne pred verziou Reactu 16.8 ako trieda, ktorá vracia takzvané JSX elementy – rozšírenie JavaScriptu. Tieto JSX elementy umožňujú zapisovať HTML štruktúru priamo v JavaScripte. Komponenty by mali opisovať vzhľad aplikácie, ale môžu obsahovať aj stav, pomocou ktorého môžeme do komponentov zakomponovať aj nejakú logiku. Ak majú komponenty stav, tak ich React sleduje a pri zmene stavu podľa potreby aktualizuje len potrebné časti rozhrania. Využitie stavu v Reacte sa realizuje pomocou tzv. **hookov** – funkcií, ktoré umožňujú pristupovať k funkcionalite Reactu v rámci funkcionálnych komponentov. Medzi najpoužívanejšie patrí napríklad hook `useState`, ktorý slúži na správu stavu komponentu.

1.3.4 Redux a Redux Toolkit

Redux [12] je JavaScriptová knižnica na centralizáciu a správu stavu. V kombinácii s knižnicou React Redux využívame Redux na sledovanie zdieľaného stavu z rôznych komponentov našej aplikácie.

Redux funguje na princípe akcií a reducerov. *Akcie* sú objekty, ktoré popisujú, aká akcia sa v aplikácii udiala (napríklad kliknutie na tlačidlo, písanie do textového poľa) pomocou typu a voliteľných parametrov (napríklad nový text). Zmenu stavu zabezpečí *reducer* – funkcia, ktorá ako argumenty dostane stav a akciu a na základe týchto argumentov zostrojí nový stav, ktorý vráti. Počas tohto procesu sa pôvodný stav nesmie zmeniť. Na prístup k aktuálnym údajom zo stavu sa využívajú *selektory* – funkcie, ktoré dostanú ako argument stav a vrátia na základe neho nejakú hodnotu.

V našej aplikácii používame aj Redux Toolkit [13], ktorý zjednodušuje prácu s Reduxom. Poskytuje napríklad funkcie `createSlice` alebo `createReducer`, ktoré umožňujú pomocou knižnice immer písať mutable kód (ktorý je interne prerobený na immutable), ktorý je často viac čitateľný, prehľadnejší a jednoduchší.

1.3.5 Vite

Vite [14] je moderný nástroj na zostavovanie aplikácie zo zdrojových kódov a vývojový server určený na vyvíjanie webových aplikácií a knižníc. Má mnoho šablón pre rôzne kombinácie webových technológií, ako sú napríklad React, Vue, Svelte alebo TypeScript, ktoré sú najčastejšie pri vývoji. Každá šablóna obsahuje minimálnu štruktúru, vhodnú na rýchle rozbehnutie vývoja. V našej práci využijeme šablónu s kombináciou React a TypeScript, ktorá je pre nás vhodná. Vite taktiež využívame na zostavovanie knižnice pre aplikáciu Logický hárok (viď nasledujúca časť).

```
1  interface PrepareResult {
2      instance: any,
3      getState: (instance: any) => any,
4  }
5  declare function prepare(initialState?: any): PrepareResult
6
7  interface AppProps {
8      instance: any,
9      isEdited: boolean,
10     onStateChange: () => void
11 }
12 declare function AppComponent(props: AppProps): JSX.Element
```

Ukážka kódu 1.1: Exportované funkcie pre Logický hárok

1.4 Súvisiace aplikácie

V tejto časti si opíšeme aplikácie, ktoré súvisia s našou aplikáciou. Prvou aplikáciou je Logický hárok, kde sú zjednotené všetky výučbové nástroje používané na predmete Logika pre informatikov na FMFI UK a druhou je predchádzajúca verzia našej aplikácie.

1.4.1 Logický hárok

Logický hárok je webová aplikácia, ktorá vznikla v rámci bakalárskej práce Mateja Moka [5]. Spája pomôcky na výučbu matematickej logiky, ktoré pôvodne vznikli ako samostatné webové aplikácie. Vyučujúci v nej vedia písať zadania a vkladať tieto aplikácie. Jednou z týchto aplikácií je aj náš Prieskumník štruktúr. Aby sa tam dali aplikácie vložiť, je potrebné, aby boli zverejnené ako balíky systému npm [15] a exportovali 2 funkcie – funkciu `prepare` slúžiacu na inicializáciu vloženej aplikácie a funkciu `AppComponent`, ktorá predstavuje funkcionálny komponent knižnice React. Deklarácie týchto funkcií môžeme vidieť v ukážke 1.1.

1.4.2 Prieskumník štruktúr

Prieskumník štruktúr je jedna z integrovaných aplikácií v Logickom hárku. Jej cieľom je pomôcť študentom s porozumením logiky prvého rádu, konkrétne splniteľnosťou formúl v doméne pri ohodnotení. Študenti si vedia vytvoriť jazyk – vymenovať individuové konštanty, predikátové a funkčné symboly. Ďalej si môžu určiť doménu a následne interpretáciu jazyka. Po vytvorení jazyka vedia študenti skúmať splniteľnosť formúl a termov tak, že zadajú formulu a následne určia svoj predpoklad – či je formula splnená,

resp. akú hodnotu má term. Pri splniteľnosti formúl majú študenti možnosť si zahrať Henkinovu-Hintikkovu hru pre hlbšie pochopenie splniteľnosti formúl. Po vypracovaní cvičenia je možné obsah exportovať a následne ho neskôr importovať. Aplikácia je výsledkom 3 bakalárskych prác – základnú verziu s množinovým pohľadom vytvoril Milan Cifra [1], grafovým pohľadom ju obohatil Miroslav Baluch [2] a nakoniec do nej pridal Richard Tóth [3] Henkinovu-Hintikkovu hru. Prieskumník štruktúr hlbšie analyzujeme v kapitole 2.

Kapitola 2

Analýza existujúceho riešenia

V tejto kapitole si vysvetlíme, ako fungovala pôvodná verzia, akú mala súborovu štruktúru a všetku jej funkcionalitu. Potom si rozoberieme jej nedostatky a problémy, ktoré budeme v ďalšej kapitole riešiť.

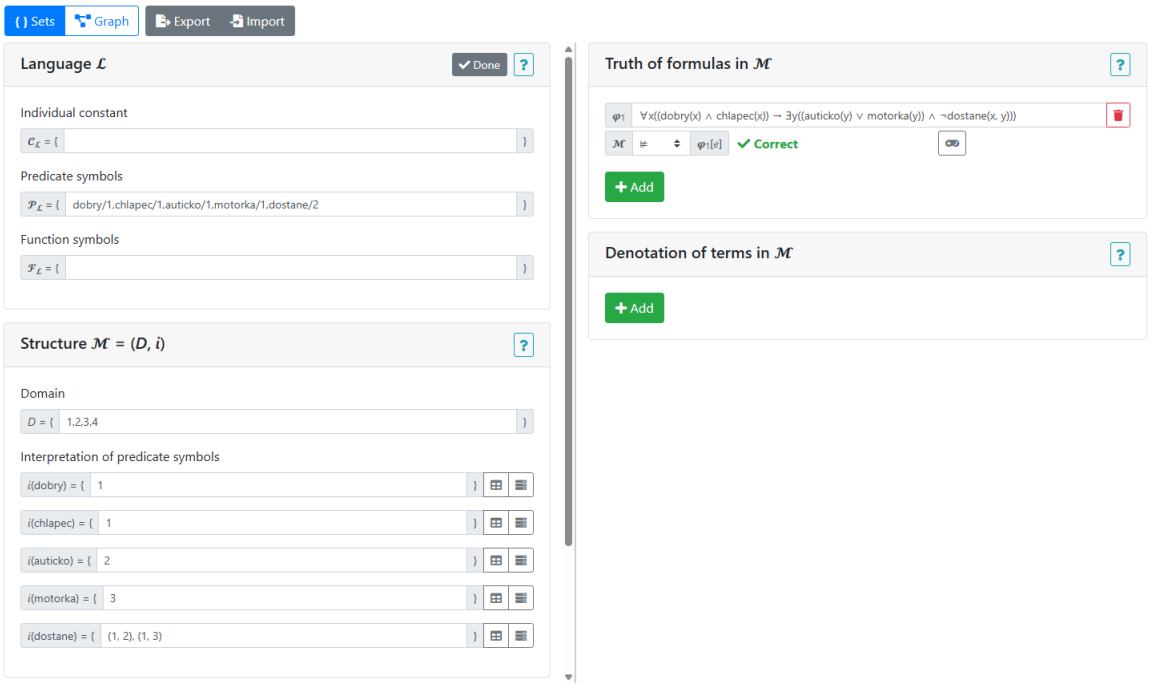
2.1 Architektúra pôvodnej aplikácie

V tejto časti sa budeme venovať analyzovaniu štruktúry a fungovaniu aplikácie.

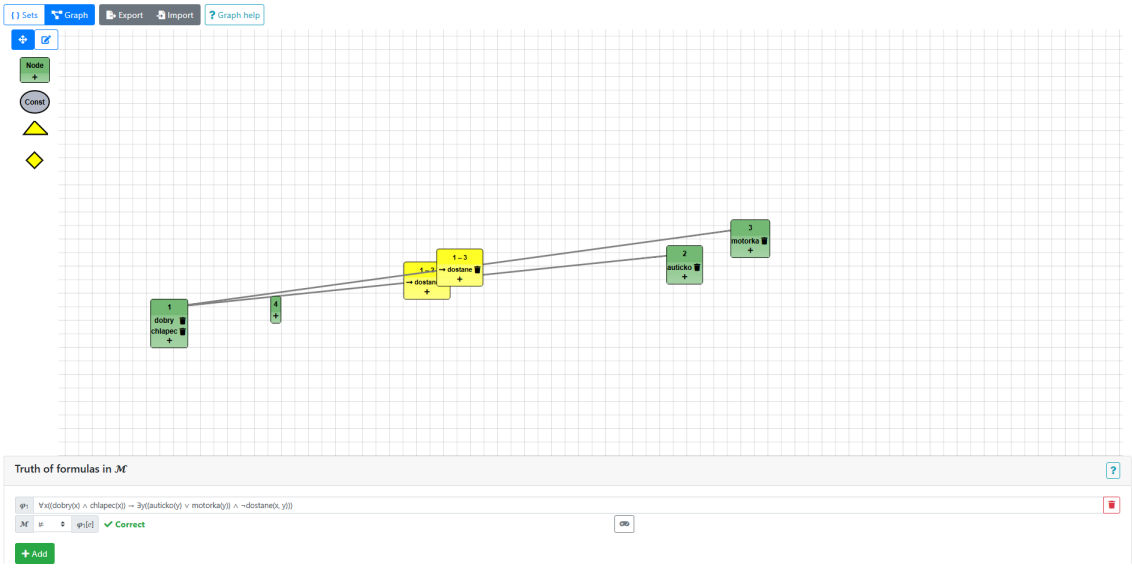
2.1.1 Funkcionalita

V tejto časti si ukážeme funkcionalitu pôvodnej aplikácie. Aplikácia je zložená z troch častí (obrázok 2.1) – hore sa nachádza panel nástrojov, vľavo máme popis definíciu konkrétneho jazyka logiky prvého rádu, definíciu konkrétnej štruktúry pre tento jazyk a konkrétneho ohodnotenia premenných. Vpravo sú samotné definície formúl a termov. Navyše v hornej oblasti každej tejto časti je aj tlačidlo `?`, ktoré po kliknutí naň zobrazí krátky popis ako funguje daná časť. V paneli nástrojov si vieme vybrať typ zobrazenia – množinové (obrázok 2.1) alebo grafové (obrázok 2.2) a máme možnosť exportovať/importovať stav aplikácie.

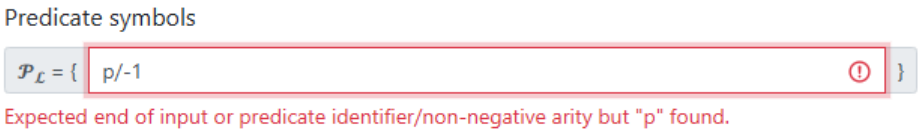
V definícii jazyka logiky prvého rádu si vieme zvoliť množiny mimologických symbolov, v definícii štruktúry jej doménu a interpretáciu mimologických symbolov. Definície spomínaných množín sa vpisujú do ich textových polí v zvyčajnej syntaxi (viď obrázok 2.1) – teda prvky oddelené čiarkami, arita sa označuje ako *názov/arita*. Ak niekde nastala chyba, ako je napríklad záporná arita (obrázok 2.3) alebo prvok nie je definovaný v doméne pri predikátových a funkčných symboloch, tak dostaneme okamžitú spätnú väzbu. Interpretácie konštánt si zvolíme tak, že rozbalíme menu, kde sú všetky prvky domény a jeden z nich si vyberieme. V prípade interpretácie predikátových a funkčných symbolov máme 3 možné pohľady – množinový pohľad, tabuľkový pohľad (obrázok 2.4) a databázový pohľad (obrázok 2.5) – ktoré sú konvertované do množinového zápisu.



Obr. 2.1: Množinové zobrazenie pôvodného Prieskumníka štruktúr



Obr. 2.2: Grafové zobrazenie pôvodného Prieskumníka štruktúr



Obr. 2.3: Spätná väzba pri zápornej arite predikátu

Interpretation of predicate symbols

$i(p) = \{$ (1, 1), (2, 2), (2, 3) $\}$

p/2	1	2	3
1	☑	☐	☐
2	☐	☑	☑
3	☐	☐	☐

Obr. 2.4: Tabuľkový pohľad interpretácie predikátu

Interpretation of predicate symbols

$i(p) = \{$ (1, 1), (2, 2), (2, 3) $\}$

m_1	m_2
1	1
2	2
2	3

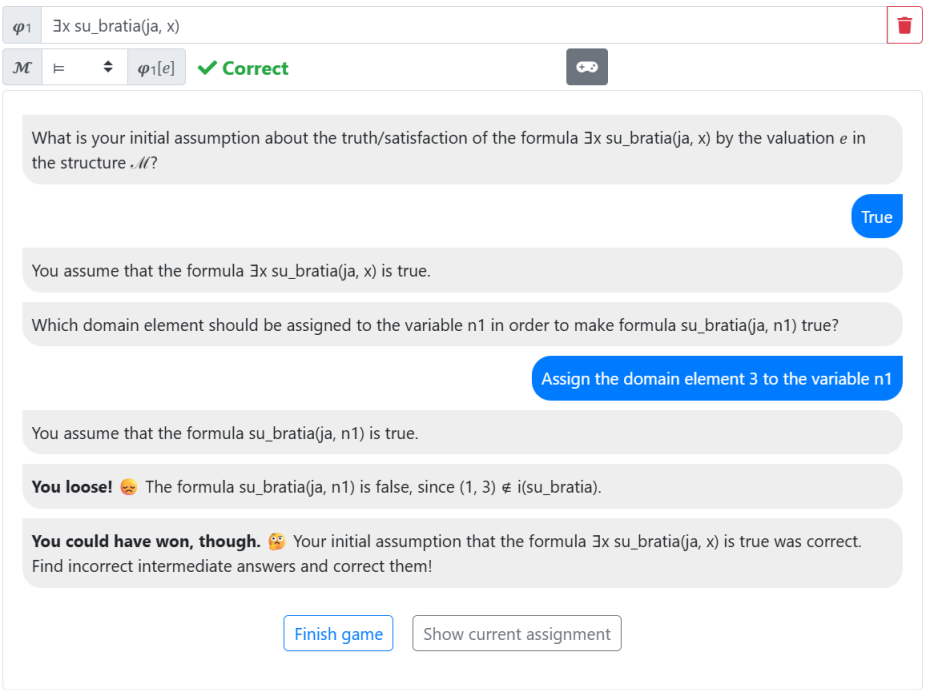
Obr. 2.5: Databázový pohľad interpretácie predikátu

Pri formulách a termoch je tiež okamžitá spätná väzba v prípade chyby, ale aj pri odhade splniteľnosti, resp. hodnoty termu. Pri formulách je aj možnosť odohrať Henkinovu-Hintikkovu hru, ktorú sme zadefinovali v časti 1.2. Jej cieľom je pomôcť študentovi pochopiť, prečo je daná formula splnená – to dosiahne pýtaním sa série otázok, kde si študent vyberá buď prvok domény, alebo jednu z formúl. Na konci hry sa študent dozvie, či vyhral, a ak nie (napr. obrázok 2.6), ale mohol, tak mu to hra oznámi a vie sa v nej vrátiť ku svojim predchádzajúcim voľbám a vybrať si inak alebo odohrať celú hru odznova. Ak vyhrať nemohol, tak má možnosť odohrať celú hru odznova. Počas hry má študent možnosť zobraziť aktuálne ohodnotenie pomocou tlačidla **Show current assignment**.

2.1.2 Štruktúra modulov zdrojového kódu

V tejto časti budeme popisovať štruktúru súborov v `/src/` kde ležia pre nás podstatné súbory. Všetky ďalšie cesty budú relatívne od priečinku `/src/`.

- **`buttons`** – V tomto priečinku sa nachádzajú Reactové komponenty všetkých tlačidiel aplikácie.
- **`constants`** – Tento priečinok obsahuje konštanty používané v celej aplikácii, ako sú napríklad správy v Henkinovej-Hintikkovej hre alebo chybové hlášky pri nesprávnom vstupe.
- **`expressions_view`** – V tomto priečinku sú všetky súbory komponentov týkajúce sa Henkinovej-Hintikkovej hry a výrazov – teda formúl a termov.



Obr. 2.6: Prehra Henkinovej-Hintikkovej hry pri správnom odhade

- **`graph_view`** – V tomto priečinku sú všetky súbory, ktoré sa týkajú komponentov grafového zobrazenia jazyka a štruktúry.
- **`math_view`** – V tomto priečinku sú všetky súbory komponentov, ktoré sa používajú v celej aplikácii.
- **`model`** – Tento priečinok obsahuje matematický model využívaný na vyhodnocovanie výrazov.
- **`redux`** – Tento priečinok obsahuje ďalšie priečinky týkajúce sa knižnice Redux.
  - `redux/actions` – Nachádzajú sa tu definované Redux akcie a ich typy ako konštanty.
  - `redux/containers` – Obsahuje Reactove komponenty, ktoré obaľujú prepojené komponenty s Reduxom pomocou staršej funkcie `connect`.
  - `redux/reducers` – Obsahuje Reduxove reducre pre celú aplikáciu.
  - `redux/selectors` – Obsahuje Reduxove selectory, ktoré vracajú inštancie jazyka, štruktúry a ohodnotenia z modelu.
- **`App.tsx`** – Hlavný Reactový komponent.
- **`AppComponent.tsx`** – Reactový komponent pre Logický hárok.
- **`index.tsx`** – Reactový komponent, ktorý sa zobrazuje na webe.

2.1.3 Dátový model na vyhodnocovanie výrazov

Trieda `Language` uchováva symboly reprezentujúce konštanty, predikátové symboly a funkčné symboly – popis jazyka. Trieda je v asociácii s triedou `Expression`, keďže každý výraz môže obsahovať len symboly definované v konkrétnom jazyku. Zároveň je asociovaná aj s triedou `Structure`, ktorá zabezpečuje interpretáciu týchto symbolov. Trieda `Structure` reprezentuje interpretáciu jazyka. Obsahuje premenné `iConstant`, `iPredicate`, `iFunction`, ktoré predstavujú časti interpretačnej funkcie a premennú `domain`, ktorá predstavuje množinu prvkov domény. Trieda `Expression` predstavuje základ pre všetky výrazy jazyka. Delí sa na dve podtriedy – `Term` a `Formula`. Z triedy `Term` sú odvodené výrazy reprezentujúce konštanty, funkcie a premenné.

Trieda `Formula` slúži ako základ pre logické formuly, medzi ktoré patria predikátové a rovnostné atómy, výrazy s logickými spojkami a kvantifikované formuly. Každý výraz je možné vyhodnotiť prostredníctvom metódy `eval()`. V prípade triedy `Term` je výsledkom prvok z domény štruktúry. V prípade triedy `Formula` ide o pravdivostnú hodnotu true alebo false. Tento model si upravíme v rámci návrhu a jeho class diagram môžeme vidieť na obrázku 3.1.

2.1.4 Štruktúra stavu

V tejto časti si rozoberieme, ako bol navrhnutý stav pôvodnej aplikácie.

- `common` – Táto časť stavu obsahuje zdieľané dáta, konkrétne `teacherMode`, kde sa dá zvoliť, či chceme mať učiteľský mód alebo nie. V učiteľskom móde je možné zamknúť niektoré vstupné textové polia ako napríklad formuly, ku ktorým hľadáme štruktúru, v ktorej sú pravdivé.
- `language` – V tejto časti stavu je popis jazyka – individuové konštanty a predikátové a funkčné symboly. Skladá sa z atribútov `constants`, `predicates`, `functions`, ktorých hodnotou sú objekty s atribútmi:
  - `value` – reťazcový obsah príslusného vstupného textového poľa,
  - `locked` – booleovská hodnota či je pole zamknuté,
  - `errorMessage` – aktuálnu chybovú hlášku pre príslušnú časť
  - `parsed` – štruktúrovanú hodnotu získanú syntaktickou analýzou reťazca `value` pre `predicates` a `functions` to je pole objektu `SymbolWithArity`, ktorý môžeme vidieť v ukážke 2.1 a pre `constants` je to pole reťazcov s názvami symbolov.
- `structure` – V tejto časti stavu sa nachádzajú dáta štruktúry, teda to je interpretácia individúových konštánt, predikátových a funkčných symbolov. Ďalej sa tu nachádza aj ohodnotenie a doména:

```
1 interface SymbolWithArity {
2         name: string,
3         arity: number
4 }
```

Ukážka kódu 2.1: Objekt SymbolWithArity vracaný parserom

  - `constants`, `predicates`, `functions` sú mapy, ktorých kľúče sú reťazce vo formáte `názov/arita` a k nim sú priradené ich interpretácie,
  - `variables` a `domain` sú objekty, ktoré obsahujú rovnaké atribúty ako premenné `constants`, `predicates` a `functions` v časti stavu pre `language`.

- `expressions` – V tejto časti stavu máme všetky formuly, termy a dáta k nim:
  - `formulas` a `terms` je pole dát k výrazom, teda objektov, ktoré obsahujú `value` – reťazec s textovou reprezentáciou výrazu, `expressionValue` – hodnota výrazu po sparsovaní a vyhodnotení, `answerValue` – predpoklad študenta o pravdivosti, `errorMessage` – chybová hláška daného výrazu, premenná `inputLocked` a `answerLocked` – informácie o tom, či je daná časť zamknutá a `parsed` – sparsovaný výraz.
- `diagramState` Táto časť stavu obsahuje tieto premenné:
  - `diagramModel` – obsahuje referencie všetkých objektov v grafe,
  - `diagramEngine` – obsahuje engine grafu vrátane informácií o canvase,
  - `domainNodes`, `constantNodes`, `ternaryNodes`, `quaternaryNodes` sú mapy, kde kľúč je meno vrcholu a hodnotou je objekt vrcholu,
  - `editableNodes` – obsahuje booleovskú hodnotu, ktorá určuje či je v grafovom pohľade zapnutý editovací alebo pohybujúci mód.

2.2 Nedostatky a problémy pôvodnej aplikácie

V tejto kapitole sa zameriame na hlavné nedostatky a problémy, ktoré budeme chcieť v našej aplikácii odstrániť.

2.2.1 Miešanie JavaScriptu a TypeScriptu

V pôvodnej verzii sa z dôvodu viacerých autorov a postupného inkrementálneho vývoja mieša využitie TypeScriptu a JavaScriptu. Ďalším dôvodom môže byť aj to, že knižnice ako Redux a React-Redux dostali oficiálnu TypeScript podporu koncom roka

2019. Takéto miešanie typovaného a netypovaného jazyka môže mať za následok veľa prehliadnutých chýb alebo iných typov, ako by sme očakávali.

2.2.2 Kombinácia funkčných komponentov a triednych komponentov

V aplikácii je už prevažne väčšia časť komponentov napísaná ako funkčné komponenty, ale nájdu sa aj miesta, ako napríklad grafový pohľad, ktoré využívajú triedne komponenty. Jednou z najväčších nevýhod triednych komponentov je absencia Reactových hookov, ako sú napríklad `useState` alebo `useEffect`. Triedne komponenty sú taktiež zastaralé a v súčasnej verzii Reactu sa už nepoužívajú.

2.2.3 Stará verzia Reduxu a absencia Redux Toolkitu

Aplikácia je napísaná prevažne v starej verzii Reduxu, kým ešte neexistoval Redux Toolkit. Bez Redux Toolkitu je viac manuálnej konfigurácie, ako napríklad akcie sa museli robiť manuálne alebo reducery spracúvajú akcie cez switch bloky. Taktiež pre zmenu stavu nie je integrovaná knižnica Immer, takže je potrebné robiť zdĺhavé kópie pomocou tzv. rozbaľovania, kde mohli nastať nežiadúce chyby.

2.2.4 Okamžitá odozva splniteľnosti formuly

Často je okamžitá odozva dobrá vec, ale v tomto prípade tomu tak nie je. Podľa pozorovania vyučujúcich – vedúceho tejto práce a jeho kolegýň a kolegov – pri okamžitom vyhodnotení zvyknú študenti sledovať len výsledok: správne/nesprávne. To ich nedonúti zamyslieť sa nad splnením formuly, ale niektorí študenti len tipujú a podľa zadania upravujú interpretácie predikátových a funkčných symbolov tak, aby im pri všetkých formulách svietila zelená „fajka“ a nápis „správne“.

2.2.5 Zriedkavé využívanie Henkinovej-Hintikkovej hry

Tento problém ide ruka v ruke s predchádzajúcou časťou 2.2.4. Tým, že hra má v aplikácií len doplnkovú úlohu, ju využíva málo študentov. Cieľom hry je ale lepšie porozumenie splneniu formúl a prepojenie jeho intuitívneho chápania s definíciou. Ak študent tejto téme rozumie, tak Henkinovou-Hintikkovou hrou rýchlo prejde a utvrdí si svoje poznatky.

Kapitola 3

Návrh novej aplikácie

V tejto kapitole si popíšeme požiadavky na reimplementovanú aplikáciu, nový návrh stavu aplikácie, dátový model na vyhodnocovanie formúl a termov a nakoniec aj na návrh implementácie Henkinovej-Hintikkovej hry pomocou označených formúl.

3.1 Požiadavky

Hlavnou požiadavkou je odstrániť nedostatky a problémy, ktoré sme opísali v kapitole 2.2. V skratke si ich pripomenieme:

- Chceme používať len TypeScript kvôli typovej podpore.
- Všetky Reactové komponenty budú funkčné komponenty.
- V Reduxe chceme používať nové funkcionality a Redux Toolkit.
- Chceme implementovať Henkinovu-Hintikkovu hru ako certifikáciu splniteľnosti formuly.

Ďalšími požiadavkami sú aj zachovanie reaktivity a stability aplikácie, prípadne ich vylepšenie a exportovanie knižničnej verzie našej aplikácie pre Logický hárok.

3.1.1 Funkcionalita

Keďže ide o reimplementáciu, je potrebné si vhodne zvoliť , ktoré časti pôvodnej aplikácie chceme ponechať. Rozhodli sme sa ponechať len množinový pohľad, keďže grafový pohľad bol študentmi málo používaný a podľa vyučujúcich je neprehľadný. Ďalej sme ponechali jazyk, v štruktúre sme urobili menšie zmeny – interpretácia konštánt sa bude zapisovať do textového poľa namiesto výberu z menu a pri interpretáciách predikátov alebo funkcií sme ponechali len množinový pohľad. Ohodnotenie sme taktiež ponechali rovnaké. Z možnosti vyhodnocovať formuly aj termy sme ponechali len formuly, keďže

vyhodnocovanie termov sa nepoužíva v žiadnych cvičeniach. Na rozdiel od starej aplikácie sa na základe požiadaviek učiteľov nebudú formuly vyhodnocovať automaticky, ale to, či je formula splnená sa bude musieť overiť Henkinovou-Hintikkovou hrou.

3.1.2 Požívateľské rozhranie

Požívateľské rozhranie sme sa rozhodli ponechať rovnaké ako v pôvodnej aplikácii, až na jednu vec – v Henkinovej-Hintikkovej hre je možnosť kroku späť stále viditeľná, ale len pri výberoch, kde dáva zmysel, na rozdiel od pôvodnej verzie, kde je viditeľná až po prejdení myšou na daný krok.

3.2 Interpretácia Henkinovej-Hintikkovej hry

Henkinovu-Hintikkovu hru sme si opísali v kapitole 1.2. Pre naše potreby si ju ale upravíme pomocou označených formúl. Ak by sme používali len pravidlá z kapitoly 1.2.1, tak by sme pri implementácii potrebovali 13 pravidiel: 4 základné pravidlá + 2 odvodené pravidlá (implikácia a ekvivalencia) + 6 otočených pravidiel pre negáciu + výherné pravidlo. Toto vieme pomocou označených formúl zredukovať na 5 pravidiel pre hru. Taktiež originálna hra rovno prepisuje výskyty premenných, zatiaľ čo my budeme využívať ohodnotenie.

3.2.1 Označené formuly

V tejto časti si popíšeme označené formuly. Všetky definície sú z poznámok k prednáškam predmetu Logika pre informatikov [6].

Nech X je formula logiky prvého rádu. Postupnosti symbolov $\mathbf{T}X$ a $\mathbf{F}X$ nazývame označenými formulami. Ak je označená formula $\mathbf{T}X$, tak to znamená, že podľa originálnych pravidiel z časti 1.2.1 vyberá daný hráč. Naopak, ak je označená formula $\mathbf{F}X$, tak si hráči roly vymenia.

Nech $\mathcal{M}$ je štruktúra pre jazyk L, e je ohodnotenie indivíduových premenných a X je formula jazyka L. Potom:

- $\mathcal{M}$ spĺňa označenú formulu $\mathbf{T}X$ pri ohodnotení e vtt $\mathcal{M}$ spĺňa formulu X pri ohodnotení e, skrátene: $\mathcal{M} \models \mathbf{T}X[e]$ vtt $\mathcal{M} \models X[e]$;

- $\mathcal{M}$ spĺňa označenú formulu $\mathbf{F}X$ pri ohodnotení e vtt $\mathcal{M}$ nespĺňa formulu X pri ohodnotení e, skrátene: $\mathcal{M} \models \mathbf{F}X[e]$ vtt $\mathcal{M} \not\models X[e]$.

3.2.2 Typy označených formúl a ich podformuly

Pomocou označených formúl vieme definovať nové pravidlá pre Henkinovu-Hintikkovu hru.

Označené formuly typu α

Označené α formuly sú popísané v tabuľke 3.1, kde A a B sú formuly, α_1 a α_2 sú ich označené podformuly.

α	α_1	α_2
$\mathbf{T}(A \wedge B)$	$\mathbf{T}A$	$\mathbf{T}B$
$\mathbf{F}(A \vee B)$	$\mathbf{F}A$	$\mathbf{F}B$
$\mathbf{F}(A \rightarrow B)$	$\mathbf{T}A$	$\mathbf{F}B$
$\mathbf{T}\neg A$	$\mathbf{F}A$	$\mathbf{F}A$
$\mathbf{F}\neg A$	$\mathbf{T}A$	$\mathbf{T}A$

Tabuľka 3.1: Označené formuly typu α

Označené formuly typu β

Označené β pravidlá sú popísané v tabuľke 3.2, kde A a B sú formuly, β_1 a β_2 sú ich označené podformuly.

β	β_1	β_2
$\mathbf{F}(A \wedge B)$	$\mathbf{F}A$	$\mathbf{F}B$
$\mathbf{T}(A \vee B)$	$\mathbf{T}A$	$\mathbf{T}B$
$\mathbf{T}(A \rightarrow B)$	$\mathbf{F}A$	$\mathbf{T}B$

Tabuľka 3.2: Označené formuly typu β

Označené formuly typu γ

Označené γ pravidlá sú popísané v tabuľke 3.3, kde A je formula, γ_1 je označená podformula, x je premenná a t je term substituovateľný za x v A.

Označené formuly typu δ

Označené δ pravidlá sú popísané v tabuľke 3.4, kde A je formula, δ_1 je označená podformula, x je premenná a y je premenná substituovateľná za x v A.

γ	γ_1
$\mathbf{T}\forall x\, A$	$\mathbf{T}A\{x \mapsto t\}$
$\mathbf{F}\exists x\, A$	$\mathbf{F}A\{x \mapsto t\}$

Tabuľka 3.3: Označené formuly typu γ

δ	δ_1
$\mathbf{T}\exists x\, A$	$\mathbf{T}A\{x \mapsto y\}$
$\mathbf{F}\forall x\, A$	$\mathbf{F}A\{x \mapsto y\}$

Tabuľka 3.4: Označené formuly typu δ

3.2.3 Nové pravidlá Henkinovej-Hintikkovej hry

Podľa typov označených formúl z časti 3.2.2 vieme vytvoriť ekvivalentnú verziu originálnych pravidiel, ktoré sme si popísali v časti 1.2.1.

Hra sa znovu odohráva medzi 2 hráčmi – Študentom a Svetom, kde sa rovnako ako v pôvodnej verzii Študent snaží formulu dokázať a Svet sa ju snaží vyvrátiť. Nech $\mathcal{L}$ je pevne daný jazyk so štruktúrou $\mathcal{M} = (D, i)$, kde D je doména a i je interpretačná funkcia. Nech e je ohodnotenie a nech A^+ je označená formula. Označme si aj krok hry pri ohodnotení ako $G(A^+, e)$. Hra má nasledujúce pravidlá:

Výherné pravidlo: ak A^+ je označená atomická formula, potom ak A^+ je splnená v štruktúre $\mathcal{M}$ pri ohodnotení e tak vyhral **Študent**, inak vyhral **Svet**.

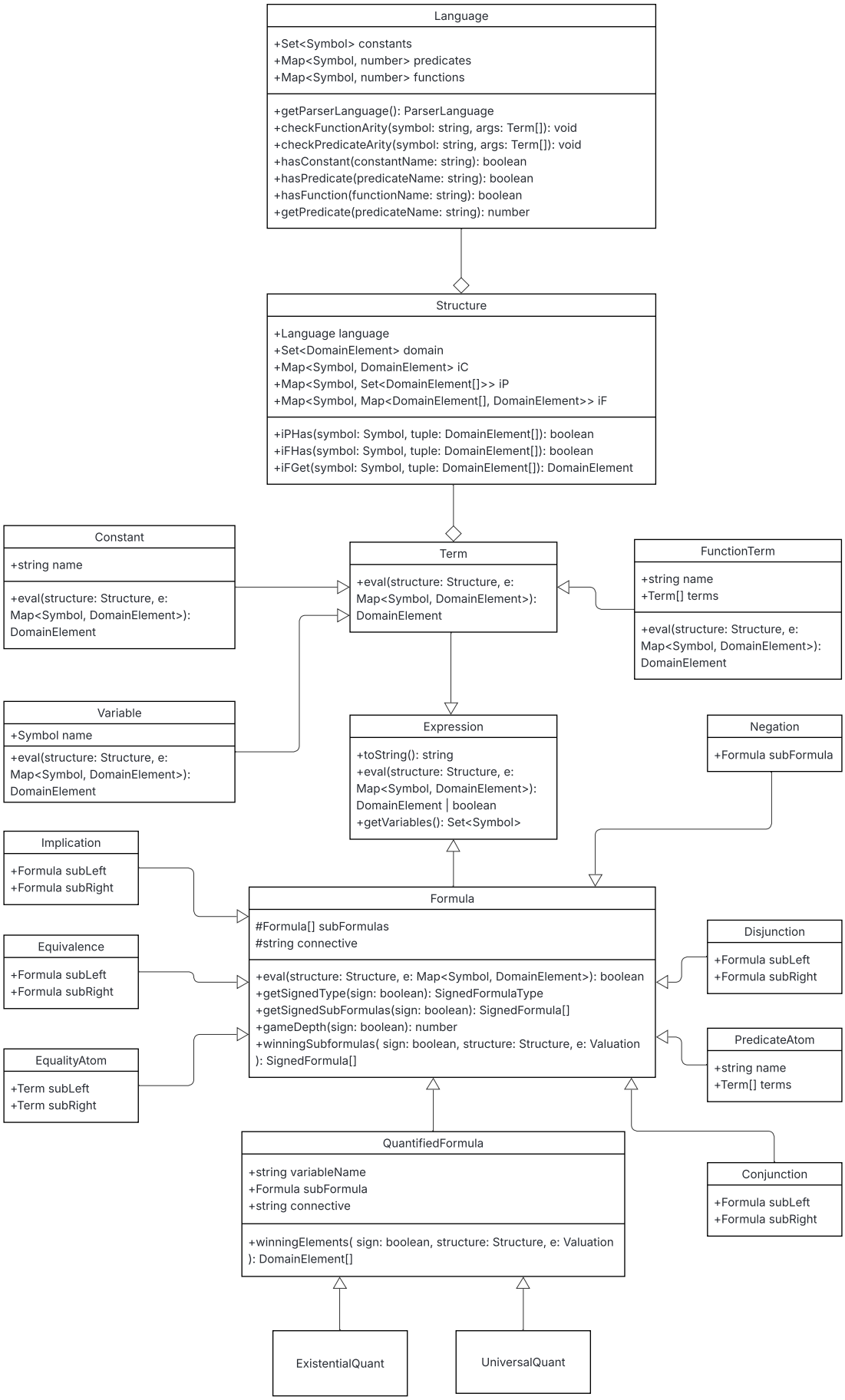
Pravidlo α: ak A^+ je typu α, tak si vyberá **Svet** jednu z možností α_1 alebo α_2 a hra pokračuje s danou možnosťou bez zmeny ohodnotenia ako $G(\alpha_i)[e]$.

Pravidlo β: ak A^+ je typu β, tak si vyberá **Študent** jednu z možností β_1 alebo β_2 a hra pokračuje s danou možnosťou bez zmeny ohodnotenia ako $G(\beta_i)[e]$.

Pravidlo γ: ak A^+ je typu γ, tak si vyberá **Svet** prvok d z domény $d \in D$ a hra pokračuje s možnosťou γ_1 so zmenou ohodnotenia ako $G(\gamma_1)[e(x/d)]$, kde x je premenná viazaná prvým kvantifikátorom vo formule A^+.

Pravidlo δ: ak A^+ je typu δ, tak si vyberá **Študent** prvok d z domény $d \in D$ a hra pokračuje s možnosťou δ_1 so zmenou ohodnotenia ako $G(\delta_1)[e(x/d)]$, kde x je premenná viazaná prvým kvantifikátorom vo formule A^+.

Tieto pravidlá sa rekurzívne opakujú, kým nenastane Výherné pravidlo. Hra začne tým, že si študent zvolí predpoklad o pravdivosti formuly a podľa neho označí A^+ ako $\mathbf{T}X$ ak si myslí, že je pravdivá a $\mathbf{F}X$ ak si myslí, že pravdivá nie je.



Obr. 3.1: Triedny diagram dátového modelu

3.3 Návrh stavu aplikácie

Náš centrálny stav bude zložený zo 4 častí – formúl, jazyka, štruktúry a ohodnotenia. V stave chceme mať len nevyhnutné údaje. Údaje, ktoré si vieme odvodiť, ako napríklad aktuálnu formulu v Henkinovej-Hintikkovej hre, v stave nepotrebujeme.

Jazyk

Jazyk si bude pamätať len text, ktorý je zadaný do príslušných textových polí konštánt, predikátových a funkčných symbolov.

Štruktúra

V stave pre štruktúru si budeme ukladať text, ktorý je zadaný do príslušných textových polí pre doménu a interpretácie jazyka – konštánt, predikátových a funkčných symbolov.

Ohodnotenie

V tejto časti stavu budeme mať uložený text na text, ktorý je zadaný študentom do textového poľa pre ohodnotenie.

Formuly

V tejto časti stavu budeme ukladať informácie o texte na parsovanie formúl, predpoklad študenta o splnení formuly a stav Henkinovej-Hintikkovej hry – ktoré formuly si študent alebo svet vybral, resp. ktorý prvok z domény priradil premennej alebo typ formuly v každom kroku hry.

3.4 Dátový model na vyhodnocovanie výrazov

V tejto časti si popíšeme dátový model, ktorý sme z veľkej časti prebrali z predchádzajúcej verzie (jeho popis je v časti 2.1.3) a pridali sme doň metódy na vyhodnocovanie Henkinovej-Hintikkovej hry vrátane reprezentácie označených formúl. Triedny diagram tohto nového modelu môžeme vidieť na obrázku 3.1. Medzi hlavné zmeny oproti pôvodnej verzii patrí obohatenie hierarchie o triedu `QuantifiedFormula`, od ktorej dedia triedy `ExistentialQuant` a `UniversalQuant`. Ďalšou zmenou je pridanie metódy `winningSubformulas`, ktorá vracia všetky výherné označené podformuly – to sú označené podformuly, ktoré musí svet vybrať, aby vyhral (to sú také označené podformuly, ktoré nie sú splnené v konkrétnej štruktúre pri konkrétnom ohodnotení) – pre danú označenú formulu a metódy `gameDepth`, ktorá vracia hĺbku označenej formuly do triedy

`Formula`. Do triedy `QuantifiedFormula` sme pridali metódu `winningElements`, ktorá vracia pole prvkov domény, ktoré si musí svet vybrať, aby vyhral – také prvky, ktoré keď pridáme do ohodnotenia, tak prestane byť ohodnotená formula splnená.

Kapitola 4

Implementácia novej aplikácie

V tejto kapitole si popíšeme súborovú štruktúru našej aplikácie, popíšeme si stav a model. Ďalej opíšeme ako sme implementovali Henkinovu-Hintikkovu hru a verziu našej aplikácie pre Logický hárok.

4.1 Inicializácia a súborová štruktúra

Na zostavenie základnej kostry aplikácie sme použili Vite, ktoré je bližšie popísané v časti 1.3.5. Nutnou podmienkou použitia Vite je inštalácia Node.js verzie aspoň 14.18 a manažér balíkov pre JavaScript, my sme využili npm. Následne si môžeme urobiť kostru projektu pomocou príkazu `npm create vite@latest structure-explorer -- --template react-ts`, alebo ak chceme využiť iné technológie, stačí použiť inú šablónu (template), prípadne použiť príkaz `npm create vite@latest` a Vite sa sériou otázok opýta na názov projektu a technológie. Štruktúra v našej aplikácii na koreňovej úrovni pozostáva z priečinku `/public`, ktorý obsahuje statické dáta a priečinku `/src`, v ktorom sú zdrojové kódy, ktoré majú nasledujúcu štruktúru, ktorá je odporúčaná v dokumentácii vývojármi Reduxu:

- `app/` – Obsahuje konfiguráciu a funkciu na vytvorenie centrálneho Reduxového stavu v súbore `app/store.ts` a definíciu silne typovaných hookov na prístup k stavu aplikácie v súbore `app/hooks.ts`.
- `components_helper` – Obsahuje Reactove komponenty využívané v celej aplikácii.
- `features/` – Obsahuje organizáciu aplikácie podľa funkcionality. Každá logicky ucelená časť má vlastný podpriečinok, ktorý obsahuje časť Reduxoveho stavu – *slice* a Reactové komponenty. Tento prístup nebol použitý v predchádzajúcej verzii aplikácie, kde boli súbory rozdelené podľa typu do priečinkov ako `components` alebo `reducers`. Náš prístup je prehľadnejší, keďže súvisiace súbory sú uložené

```
1 interface FormulaState {
2     text: string;
3     guess: boolean | null;
4     gameChoices: {
5     formula?: 0 | 1;
6     element?: string;
7     type: "alpha" | "beta" | "gamma" | "delta";
8     }[];
9 }
```

Ukážka kódu 4.1: Interface FormulaState

spolu, čo uľahčuje orientáciu v zdrojových kódoch. V našom prípade budeme mať 6 častí - formulas, game, import, language, structure a variables.

- `model/` – Obsahuje súbory dátového modelu, ktorý je opísaný v časti 3.4.

4.2 Dátový model na vyhodnocovanie výrazov

Z pôvodnej aplikácie sme prevzali model, ktorý bol pôvodne v JavaScripte. Z tohto modelu sme odstránili metódy, ktoré nebudeme používať, ako napríklad metódy, ktoré sa využívali v Henkinovej-Hintikkovej hre – `substitute` alebo `createCopy`. Zvyšný kód sme prepísali do TypeScriptu a implementovali sme metódy, ktoré sme popísali v časti 3.4.

4.3 Stav aplikácie

V tejto časti popíšeme štruktúru stavu aplikácie.

4.3.1 Stav formulas

Táto časť stavu sa nachádza v súbore `/src/features/formulas/formulasSlice.ts`. Obsahuje pole formúl typu `FormulaState` `allFormulas` (ich rozhranie môžeme vidieť na ukážke 4.1), ktoré obsahujú nasledujúce premenné:

- `text` – reťazec predstavujúci vstup, ktorý zadal študent,
- `guess` – boolovská hodnota reprezentujúca predpoklad študenta o pravdivosti formuly, hodnota `null` znamená, že zatiaľ neodpovedal,
- `gameChoices` – pole objektov, ktoré obsahujú:

- `formula?` – voliteľná hodnota predstavujúca index vybranej podformuly v hre pre typ β,
- `element?` – voliteľný reťazec reprezentujúci výber konkrétneho prvku domény pre typ δ,
- `type` – reťazec určujúci typ formuly (α, β, γ alebo δ).

4.3.2 Stav language

Táto časť stavu sa nachádza v súbore `/src/features/language/languageSlice.ts`. Uchováva reťazce popisujúce jazyk – premenné `constants`, `predicates` a `functions` obsahujú text zadaný študentom.

4.3.3 Stav structure

Táto časť stavu sa nachádza v súbore `/src/features/structure/structureSlice.ts`. Uchováva informácie o štruktúre interpretácie. Obsahuje premennú `domain` – reťazec reprezentujúci doménu štruktúry zadanú študentom – a objekty typu `Record<string, InterpretationState>` `iC`, `iP` a `iF`, ktoré reprezentujú interpretácie indivíduových konštánt, predikátových symbolov a funkčných symbolov zadaných študentmi.

4.3.4 Stav variables

Táto časť stavu sa nachádza v súbore `/src/features/variables/variablesSlice.ts`. Uchováva premennú `text` – reťazec reprezentujúci popis ohodnotenia zadaného študentom.

4.3.5 Reaktivita a práca so stavom

Aplikácia je postavená na veľmi reaktívnom princípe, pričom každý komponent reaguje na zmeny stavu pomocou hookov `useAppSelector` a `useAppDispatch`. Výhodou tohto prístupu je oddelenie logiky od vzhľadu a automatická reaktivita komponentov pri zmene dát. Na odvodzovanie údajov zo stavu používame selektory, prípadne memoizované selektory, ktoré obmedzujú prekresľovanie komponentov pri zmene nesúvisiacich častí stavu.

4.4 Implementácia Henkinovej-Hintikkovej hry

Najkomplexnejšou časťou aplikácie je implementácia Henkinovej-Hintikkovej hry, ktorá umožňuje študentom interaktívne overovať pravdivosť formúl. Herná logika je implementovaná v súboroch umiestnených v priečinku `/src/features/formulas`, keďže hra

môže byť spustená samostatne pre každú formulu. Komponenty zodpovedné za vizuálnu časť hry sa nachádzajú v priečinku `/src/features/game`. Na spustenie hry musí byť zadaná formula bez syntaktickej chyby a študent musí zvoliť predpoklad o pravdivosti formuly (označenie ako pravdivá alebo nepravdivá).

4.4.1 Selektory pre Henkinovu-Hintikkovu hru

Dáta potrebné pre zobrazenie a priebeh hry sú získavané pomocou memoizovaného selektora `selectHistoryData`, ktorý je vytvorený pomocou funkcie `createSelector`. Táto funkcia umožňuje kombinovať viacero selektorov do jedného. Medzi ne patrí aj `selectEvaluatedFormula`, ktorý vracia objekt s nasledujúcimi atribútmi:

- `evaluated` – booleovská hodnota určujúca, či je formula pravdivá,
- `formula` – formula z dátového modelu aplikácie,
- `error` – informácia o prípadnej chybe pri spracovaní formuly.

Výsledný selektor `selectHistoryData` vracia pole objektov typu, ktorý môžeme vidieť v ukážke 4.3, kde každý objekt obsahuje:

- `sf` – aktuálnu označenú formulu,
- `valuation` – aktuálne ohodnotenie premenných,
- `type` – typ označenej formuly (α, β, γ, δ),
- `winFormula?` – pri formule typu α obsahuje označenú formulu, ktorá je výherná,
- `winElement?` – pri formule typu γ obsahuje výherný prvok domény.

Toto pole vytvárame iteráciou cez pole výberov, ktoré sa zatiaľ v hre vykonali `state.formulas.allFormulas[id].gameChoices`, kde `id` predstavuje index formuly, ku ktorej daná hra patrí. Z memoizovaného selektora `selectEvaluatedFormula` získavame počiatočnú formulu, s ktorou pokračujeme v hre. Na základe jej typu (α, β, γ, δ) a volieb študenta následne odvodzujeme dôležité informácie:

- aktuálnu označenú formulu,
- typ formuly,
- aktuálne ohodnotenie premenných,
- ak ide o formulu typu α – pomocou funkcie `winningSubformulas` z nášho modelu získavame výhernú podformulu pre svet (vždy prvý prvok v poli),

```
1 {
2     values: string[];
3     type: "alpha" | "beta" | "gamma" | "delta";
4     subformulas?: SignedFormula[];
5     variableName?: string;
6 }
```

Ukážka kódu 4.2: Typ selektora selectGameButtons

- ak ide o formulu typu γ – pomocou funkcie `winningElements` z nášho modelu získavame výherný prvok domény pre svet (tiež prvý prvok v poli).

Na generovanie dát pre tlačidlá, ktorými študent komunikuje s hrou používame selektor `selectGameButtons`, ktorý je vytvorený pomocou funkcie `createSelector`. Typ návratovej hodnoty tohto selektora môžeme vidieť v ukážke 4.2. Na základe tohto objektu vykresľujeme interaktívne tlačidlá pre hru. Ich komponenty sú umiestnené medzi pomocnými komponentmi v priečinku `/src/components_helper`. Selektor `selectGameButtons` získava aktuálnu označenú formulu z memoizovaného selektora `selectCurrentGameFormula` a na základe jej typu priraďuje príslušné hodnoty do premenných:

- `values` – pole hodnôt, ktoré sa zobrazia ako text na tlačidlách. Ak ide o formulu typu α alebo γ, zobrazuje sa len tlačidlo `Continue`, pretože je na ťahu hráč Svet.
- `type` – typ označenej formuly (α, β, γ, δ).
- `subformulas?` – pole označených podformúl (len pre formuly typu α a β).
- `variableName?` – názov premennej (len pre formuly typu γ a δ).

4.4.2 Komponenty Henkinovej-Hintikkovej hry

Pomocou selektorov `selectHistoryData` a `selectGameButtons` vieme vytvoriť komponent zobrazujúci históriu hry – `GameHistory` a komponent pre tlačidlá ovládajúce ďalší krok – `GameControls`.

`GameHistory`

Komponent `GameHistory` slúži na vykresľovanie Henkinovej-Hintikkovej hry vo forme dialógových bublín. Využíva dáta zo selektora `selectHistoryData`, ktorý obsahuje zoznam krokov, ktoré sa v hre odohrali. Pre každý krok vytvárame konkrétnu správu,

ktorá reprezentuje buď Študentov predpoklad, správu hry, výber z možností, alebo výsledok – Študent vyhral alebo prehral. Iterujeme cez objekty histórie, pričom na základe typu formuly (α, β, γ, δ) vypisujeme správy. V prípade, že si Študent vyberal podformulu alebo prvok domény sa zobrazí aj možnosť vrátiť sa späť a výber zmeniť. Táto funkcionalita je implementovaná pomocou tlačidla, ktoré volá `reducer` – `gameGoBack` a umožňuje hráčovi zmeniť svoju voľbu. Komponent taktiež zabezpečuje automatické posúvanie pohľadu na poslednú správu pomocou referencie.

`GameControls`

Úlohou komponentu `GameControl` je zobrazovanie tlačidiel s voľbami pre Študenta počas Henkinovej-Hintikkovej hry. Na základe dát zo selektora `selectGameButtons` vytvárame príslušné tlačidlá podľa aktuálneho typu formuly (α, β, γ, δ). Tento selektor vracia informácie o možných voľbách hráča, ktoré potom zobrazujeme ako rôzne tlačidlá – `ChoiceBubble` pri výbere formúl alebo `SelectBubble` pri vyberaní prvku domény.

Komponent funguje nasledovne:

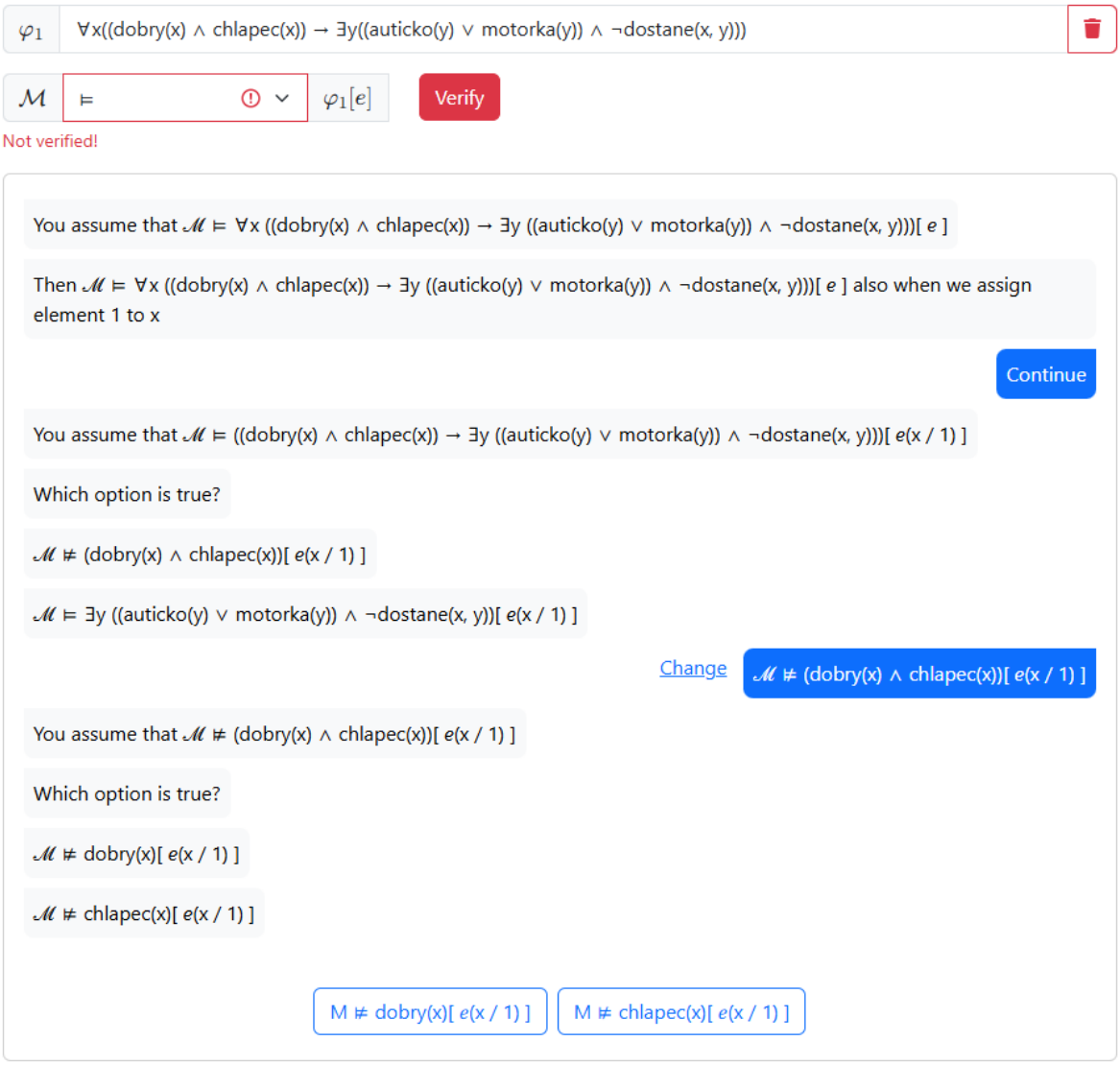
- Pri α a β formulách generujeme 2 tlačidlá s podformulami. Pri kliknutí na možnosť sa zavolá daný `reducer` – `addAlpha` alebo `addBeta`, ktorý vloží výber do stavu.
- Pri γ a δ formulách ide si hráč vyberá prvok domény. Pri kliknutí na prvok sa taktiež zavolá daný `reducer` – `addGamma` alebo `addDelta`.

Na obrázku 4.1 môžeme vidieť priebeh Henkinovej-Hintikkovej hry. Študentov predpoklad je, že formula nie je pravdivá. Prebehli už 2 kolá - najprv si vyberal svet, následne si vyberal študent a momentálne je na rade znova študent. Na výber má 2 podformuly, jeho možnosti môžeme vidieť v dolnej časti obrázka.

4.4.3 Reducery pre Henkinovu-Hintikkovu hru

Medzi reducery pre Henkinovu-Hintikkovu hru patria:

- `addAlpha`, `addBeta` – reducery reagujú na akciu typu `formulas/typFormuly`. Ako parameter dostanú objekt, ktorý obsahuje `id` formuly, ktorej hra patrí a `formula` – index formuly, ktorá bola vybraná. Na akciu reagujú tak, že do `state.formulas.allFormulas.gameChoices` pridajú nový prvok s indexom formuly a typom formuly.
- `addGamma`, `addDelta` – rovnako ako predchadzajúce reducery reagujú na akciu `formulas/typFormuly`. Ako parameter dostanú objekt, ktorý obsahuje `id` formuly, ktorej hra patrí a `element` – reťazec, ktorý bola vybraný ako prvok z do-



Obr. 4.1: Voľby v Henkinovej-Hintikkovej hre

mény. Na akciu reagujú tak, že do `state.formulas.allFormulas.gameChoices` pridajú nový prvok s vybraným prvkom a typom formuly.

- `gameGoBack` – reducer reaguje na akciu `formulas/gameGoBack` ako parameter dostane objekt, ktorý obsahuje `id` formuly, ktorej patrí hra a `index` - miesto, do akého bodu sa bude hra resetovať. Na akciu reaguje tak, že v poli `state.formulas.allFormulas.gameChoices` ponechá prvky do indexu, ktorý dostane.

4.5 Implementácia knižnice pre Logický hárok

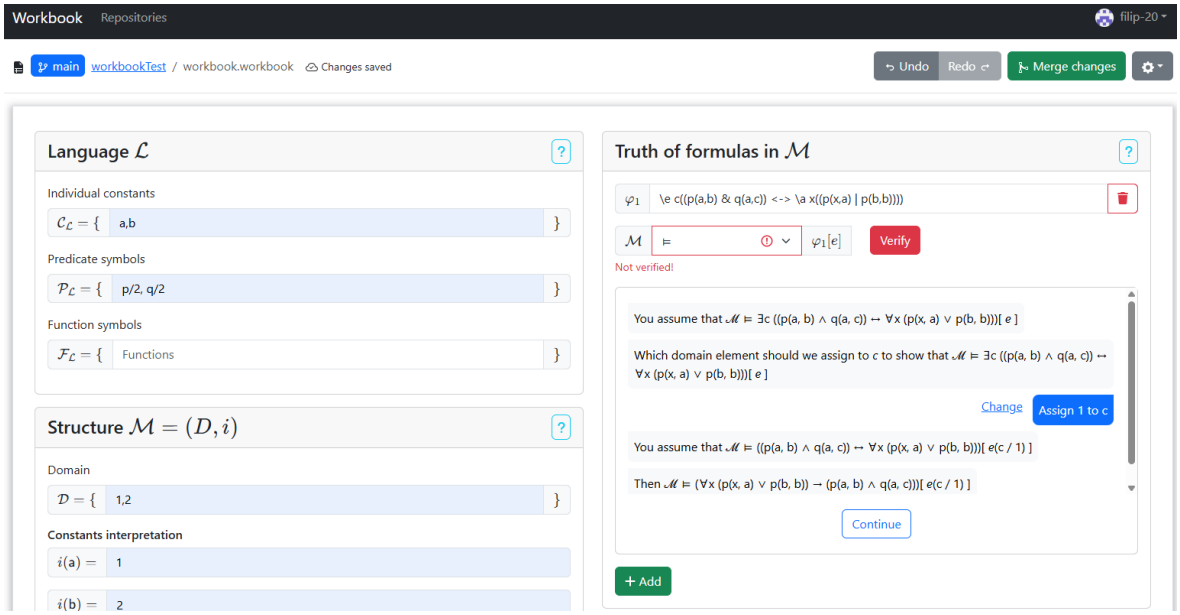
Medzi požiadavkami sme mali aj implementáciu knižničného rozhrania pre Logický hárok. V tejto časti si opíšeme implementáciu tohto rozhrania. Aby sa dala naša aplikácia vložiť do Logického hárku, musí exportovať funkcie, ktoré sú popísané na ukážke 1.1.

Funkcia `prepare`

Funkcia `prepare` slúži na inicializáciu vloženej aplikácie. Vyrobíme si v nej inštanciu reduxoveho úložiska stavu, ktorý uložíme v premennej `instance`, ktorá slúži na účely vloženej aplikácie. Vo funkcii `getState` vraciame objekt, ktorý vytvoríme pomocou `JSON.stringify`, kde pošleme celý náš stav, takže `formulas`, `language`, `structure` a `variables`. Táto reprezentácia stavu sa do Logického hárku uloží a je z nej možné obnoviť stav pri ďalšom otvorení daného hárku. Funkcia `prepare` má ako nepovinný parameter `initialState`, ktorý predstavuje počiatočný stav – ak je k dispozícii, tak zavoláme reducer `importAppState`, ktorý sa postará o načítanie stavu. Parameter `initialState` nám odovzdá Logický hárok, pri inicializácií, ak už bola naša aplikácia vložená.

Funkcia `AppComponent`

Táto funkcia predstavuje funkcionálny Reactový komponent. Naša vložená aplikácia musí notifikovať Logický hárok v prípade zmien pomocou funkcie `onStateChange`. Túto notifikáciu sme vyriešili tak, že ak nastane nejaká zmena v stave našej aplikácie, tak zavoláme funkciu `onStateChange`. V rámci samotného komponentu využívame hlavný komponent našej aplikácie `App`, ktorý je obalený reduxovym komponentom `Provider`, s parametrom `store`, ktorý sprístupňuje reduxove úložisko centrálneho stavu všetkým obaleným komponentom. Na obrázku 4.2 môžeme vidieť úspešné vloženie našej aplikácie do lokálneho Logického hárku.



Obr. 4.2: Naša aplikácia v lokálnom Logickom hárku

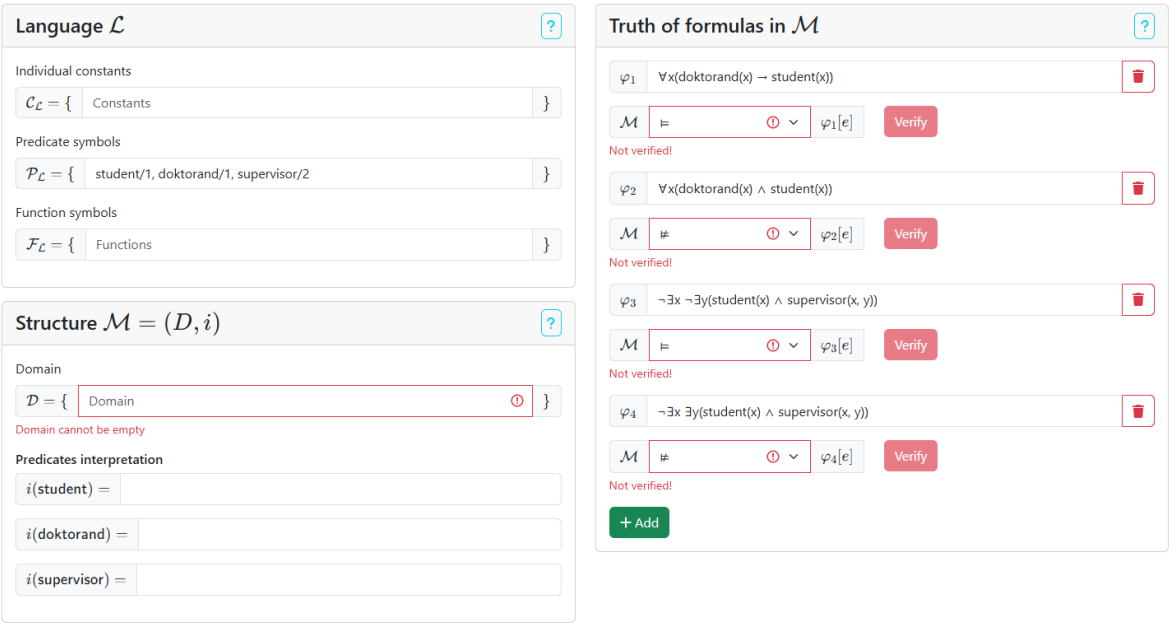
```
{
    sf: SignedFormula;
    valuation: Map<string, string>;
    type: "alpha" | "beta" | "gamma" | "delta";
    winFormula?: SignedFormula;
    winElement?: string;
}
```

Ukážka kódu 4.3: Typ selektora selectHistoryData

Kapitola 5

Testovanie

Pretože sa nám nepodarilo otestovať aplikáciu počas výučby predmetu Logika pre informatikov, zorganizovali sme menšie testovanie použiteľnosti, kde sa zapojili 3 študenti, ktorí reprezentovali cieľovú skupinu používateľov aplikácie. Všetci v minulosti používali predchádzajúcu verziu aplikácie, takže s novou verziou boli celkom oboznámení. Za úlohu (obrázok 5.1) dostali nájsť štruktúru, v ktorej sú formuly pravdivé a následne mali správnosť nájdenej štruktúry verifikovať pomocou Henkinovej-Hintikkovej hry. Formuly už dostali predvyplnené s určeným predpokladom. Pri testovaní sme sledovali hlavne pripomienky účastníkov, keďže cieľom testovania bolo zistiť hlavne používateľskú skúsenosť, prípadne odhaliť skryté chyby.



Obr. 5.1: Prieskumník štruktúr s predvyplnenými formulami zo zadania

Počas testovania sa väčšinou študenti skôr sťažovali na samotnú úlohu ako na aplikáciu. Všetky pripomienky boli smerované na Henkinovu-Hintikkovu hru. Jeden študent chcel zadať celú štruktúru naraz, pretože si myslel, že sa hra bude resetovať pri každej

zmene štruktúry a odohral všetky hry naraz. Ďalší zo študentov práveže naopak, upravoval štruktúru hlavne podľa výslednej hlášky hry. Ten istý študent sa snažil hru spustiť odznova, namiesto zmeny svojej voľby. Dvom študentom by vyhovovala možnosť prepnutia hry do „čitateľnejšieho“ módu – toho, ktorý bol v predchádzajúcej verzii, ktorý môžeme vidieť na obrázku 2.6. V rámci testovania sa našla aj chyba pri vykresľovaní tlačidiel na výber formuly – chýbalo tam aktuálne ohodnotenie. 2 z 3 študentov hru považovali za otravnú a uprednostňovali okamžité vyhodnocovanie. Posledný študent sa počas testovania na hru nijako nesťažoval, takže mu asi nevadila.

Po testovaní každý študent vyplnil krátky dotazník nazývaný System Usability Scale [16], ktorý môžeme vidieť na obrázku 5.2. Dotazník pozostáva z 10 výrokov, na ktoré používateľ odpovedá na škále od 1 (vôbec nesúhlasím) po 5 (úplne súhlasím). Dotazník poskytuje celkové číslo vyjadrujúce mieru použiteľnosti (od 0 do 100) – tzv. SUS skóre. Tento dotazník sme využili na vyhodnotenie použiteľnosti našej aplikácie. Naše SUS skóre vyšlo 80, 77.5 a 85, priemerne 80.8, čo je dobré skóre. Výsledkom bola všeobecná spokojnosť, dizajn bol podľa študentov konzistentný a aplikácia bola jednoduchá na používanie.

Ku každej otázke napíšte číslo od 1 do 5, ako veľmi súhlasíte s daným výrokom, kde: 1 vôbec nesúhlasím, 5 úplne súhlasím

1. Myslím si, že by som tento systém chcel(a) používať často. .....
2. Myslím si, že je systém zbytočne zložitý. .....
3. Myslel(a) som si, že je systém jednoduchý na používanie. .....
4. Myslím si, že by som potreboval(a) pomoc technickej osoby na používanie tohto systému. .....
5. Myslím si, že rôzne funkcie tohto systému sú dobre integrované. .....
6. Myslel(a) som si, že je v systéme veľa nekonzistencie. .....
7. Vedel(a) som si predstaviť, že väčšina ľudí sa naučí používať tento systém veľmi rýchlo. .....
8. Myslím si, že systém je ťažkopádny na používanie. .....
9. Pri používaní systému som sa cítil(a) veľmi sebavedomo. .....
10. Potreboval(a) som sa naučiť mnoho vecí predtým, ako som mohol(a) začať pracovať s týmto systémom. .....

Obr. 5.2: Dotazník na vyhodnotenie použiteľnosti

Záver

V tejto práci sme vytvorili novú verziu webovej aplikácie Prieskumník štruktúr, ktorá slúži na pomoc pri pochopení pravdivosti formúl v konkrétnej štruktúre pre jazyk logiky prvého rádu za pomoci nových verzií frameworkov, ktoré poskytujú lepšiu funkcionalitu ako pri predchádzajúcej verzii. Taktiež sa nám podarilo posilniť úlohu Henkinovej-Hintikkovej hry odstránením okamžitej odozvy pravdivosti formuly v konkrétnej štruktúre. Študenti v novej verzii musia prejsť Henkinovou-Hintikkovou hrou, aby sa dozvedeli, či je formula pravdivá.

Spravili sme analýzu pôvodnej aplikácie, teda sme analyzovali jej funkcionalitu, návrh jej centralizovaného stavu a dátového modelu a vypísali sme nedostatky, ktoré budeme chcieť vo výslednej verzii odstrániť, ako napríklad používanie JavaScriptu a TypeScriptu zároveň alebo kombináciu funkčných komponentov s triednymi komponentmi.

Navrhli sme požiadavky na funkcionalitu a používateľské rozhranie pre novú aplikáciu na základe podnetov z analýzy. Definovali sme interpretáciu Henkinovej-Hintikkovej hry pomocou označených formúl a následne sme si odvodili nové pravidlá. Navrhli sme aj centrálny stav aplikácie a dátový model.

Popísali sme si novú súborovú štruktúru a následne sme implementovali požiadavky z návrhu a taktiež aj centrálny stav a dátový model našej aplikácie. Vybrali sme si zložitejšiu časť implementácie – realizáciu Henkinovej-Hintikkovej hry, v ktorej sme si popísali, aké selektory a komponenty používame. Ďalej sme implementovali verziu našej aplikácie, ktorá sa dá vložiť do Logického hárku.

Nakoniec novú verziu aplikácie otestovali 3 študenti, ktorí už v minulosti používali predchádzajúcu verziu aplikácie a dali sme im vyplniť krátky dotazník na vyhodnotenie použiteľnosti našej aplikácie.

Aplikácia by sa dala vylepšiť v ďalších bakalárskych prácach pridaním ďalšej funkcionality pôvodnej verzie, ako napríklad pridaním grafového pohľadu alebo pridať možnosť zapisovať interpretácie pomocou tabuľky alebo databázovo. Ďalším zlepšením by mohlo byť pridanie možnosti prepínania sa medzi formálnym zápisom dialógu v Henkinovej-Hintikkovej hre, ktorý sme implementovali a menej formálnym, ktorý bol v predchádzajúcej verzii. Taktiež by mohlo byť prínosné mať možnosť nahradiť všetky logické symboly vo formulách ich príslušnými unikódovými znakmi pre ľahšiu čitateľ-

nosť.

Literatúra

[1] Milan Cifra. Prieskumník sémantiky logiky prvého rádu, 2018. Bakalárska Práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky.

[2] Miroslav Baluch. Prieskumník grafových štruktúr pre logiku prvého rádu, 2020. Bakalárska Práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky.

[3] Richard Tóth. Henkinova-hintikkova hra v prieskumníku štruktúr, 2021. Bakalárska Práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky.

[4] Jaakko Hintikka. *The game of language: Studies in game-theoretical semantics and its applications, volume 22 of Studies in Linguistics and Philosophy.* Springer Science Business Media, 1983.

[5] Matej Mok. Interaktívny pracovný zošit pre výučbu logiky pre informatikov, 2022. Bakalárska Práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky.

[6] Ján Kľuka and Jozef Šiška. Prednášky z lpi 2016/17, 2017. [Citované 2025-04-22] Dostupné z `https://dai.fmph.uniba.sk/upload/104/3/30/Lpi-prednasky-2016-17.pdf`.

[7] Vítězslav Švejdar. *Logika: neúplnost, složitost a nutnost.* Praha: Academia, 2002.

[8] HTML. Html, 2025. [Citované 2025-06-04] Dostupné z `https://html.spec.whatwg.org/multipage/`.

[9] W3C Working Group. Css snapshot 2024, 2025. [Citované 2025-06-04] Dostupné z `https://www.w3.org/TR/css-2024/`.

[10] Microsoft. Typescript documentation, 2025. [Citované 2025-06-04] Dostupné z `https://www.typescriptlang.org/docs/`.

[11] Meta Platforms, Inc. Quick start, 2025. [Citované 2025-06-04] Dostupné z `https://react.dev/learn`.

[12] Dan Abramov and the Redux documentation authors. Getting started with redux, 2024. [Citované 2025-06-04] Dostupné z `https://redux.js.org/introduction/getting-started`.

[13] Dan Abramov and the Redux documentation authors. Getting started with redux toolkit, 2023. [Citované 2025-06-04] Dostupné z `https://redux-toolkit.js.org/introduction/getting-started`.

[14] VoidZero Inc. Vite Contributors. Getting started, 2024. [Citované 2025-06-04] Dostupné z `https://vitejs.dev/`.

[15] npm. npm docs, 2025. [Citované 2025-06-04] Dostupné z `https://docs.npmjs.com/`.

[16] John Brooke. Sus: A quick and dirty usability scale. *Usability Eval. Ind.*, 189, 11 1995.